# AI Enabled Productivity Apps

Token Buffet converts flat-rate command-line interface (CLI) subscriptions, such as Claude Code, Codex CLI, Grok Build, and Gemini Antigravity (`agy`), into a local API that speaks the OpenAI, Ollama, Anthropic, and Gemini wire formats. Downstream apps and CLI agents talk HTTP to one local address, and Token Buffet shells out to each vendor's logged-in CLI, so no per-token API keys or usage bills are involved.

The whole system is config-driven: adding or changing a provider means editing a declarative `registered_providers/<name>/config.json`, and the Python engine has no provider-specific branches.

## 1. Three Long-Running Processes

All services bind to `127.0.0.1`, never `localhost`.

- **CLI Provider Proxies (ports 8000 to 8003+)**: One proxy per provider (Grok 8000, Claude 8001, Codex 8002, Antigravity 8003). Each proxy turns chat requests into CLI invocations and parses the CLI output back into chat chunks.
- **Smart Router (port 8090)**: The single endpoint clients point at. It maps a semantic tier to provider backends and balances load across subscriptions.
- **Control Center Dashboard (port 8080)**: A status and configuration view of the fleet, quotas, models, apps, router settings, and request activity.

## 2. Semantic Tiers and Routing Modes

Clients request one of three tiers instead of a vendor model string: `fast` (Quick), `medium` (Balanced), or `reasoning` (Best).

- **Usage mode**: Routes a tier to the backend with the most remaining quota, skipping depleted or stopped backends.
- **Custom mode**: Ignores quota and follows a global `custom_order` provider preference list.
- **Failover**: Failures before streaming starts advance to the next candidate, and an unresolvable request falls back to the medium tier or returns HTTP 503.

## 3. Monitoring

An extractor subsystem reads each vendor CLI's interactive `/usage` and model screens through a pseudo-console, since no API exposes them. Each proxy publishes its usage, models, and run status under `data/providers_status/`. The router polls each running proxy's `/usage` and `/models` endpoints, and the dashboard shows the results as quota gauges and status dots.

## 4. Apps and Clients

- **Integrated apps**: Obsidian (the karpathywiki plugin) and Claudia are registered through a three-layer configuration cascade from `integrations/` to `registered_apps/`.
- **Chatbot CLIs**: Claude Code, Codex CLI, Grok Build, and Gemini Antigravity run in isolated workspaces pointed at the router.

- **Consumer software**: Any app with a custom OpenAI or Ollama endpoint field can connect with a base URL of `http://127.0.0.1:8090` or `http://127.0.0.1:8090/v1`, a tier as the model name, and a dummy API key.

# Core Architecture

Token Buffet runs as three long-running process types, all on loopback `127.0.0.1`, with an extractor subsystem feeding them status data.

## 1. The Three Process Layers

- **CLI Provider Proxies (ports 8000 to 8003+)**: Built on Python's standard library `http.server`, one proxy per vendor CLI (Grok 8000, Claude 8001, Codex 8002, Antigravity 8003). A proxy binds its port first, then reads its own usage and models in a background thread.
- **Local Router (port 8090)**: The central gateway. It accepts OpenAI, Ollama, Anthropic, and Gemini requests, resolves a semantic tier to a backend, and forwards to the live port each proxy published. A standalone router launch defaults to port 11434, while `tbuffet start` binds it to 8090.
- **Dashboard (port 8080)**: Also standard library only, with static files in `dashboard/static/`. It renders whatever it finds on disk and contains no hardcoded provider or app logic.

## 2. The Extractor Subsystem

The extractor drives each vendor CLI through a pseudo-console to capture its `/usage` and model screens, then parses the saved text with the provider's configured regexes. Results are exported asynchronously to `data/providers_status/` as `<provider>_usage.json`, `<provider>_models.json`, and `<provider>_run_status.json`, so live requests never wait on a scrape.

## 3. Data Flow

- **Proxies** publish their bound port and state to `<provider>_run_status.json`.
- **Router** polls each running proxy's `/usage` and `/models` endpoints at startup and every `--refresh-interval` seconds (default 1,800).
- **Dashboard** reads the exported status files to draw gauges, status dots, and model tables.

## 4. Reliability

Non-streaming requests are buffered, spilling to disk above 1 MiB so failed attempts can be retried. The router only uses a provider marked running whose port is listening, so stale files after a forced stop are ignored.

## Dashboard: Port 8080

In the architecture of the Token Buffet AI Subscription Hub, the Control Center Dashboard operating on port 8080 serves as the central operator console and management interface. While the CLI proxies handle vendor communications and the Smart Router Gateway (port 8090) directs prompt traffic, the port 8080 dashboard provides a unified UI to monitor fleet health, manage app registrations, inspect quotas, and configure system routing.

Served at http://127.0.0.1:8080/ via python -m tokenbuffet.dashboard or the tbuffet start fleet launcher, the dashboard is strictly restricted to local loopback connections for security.

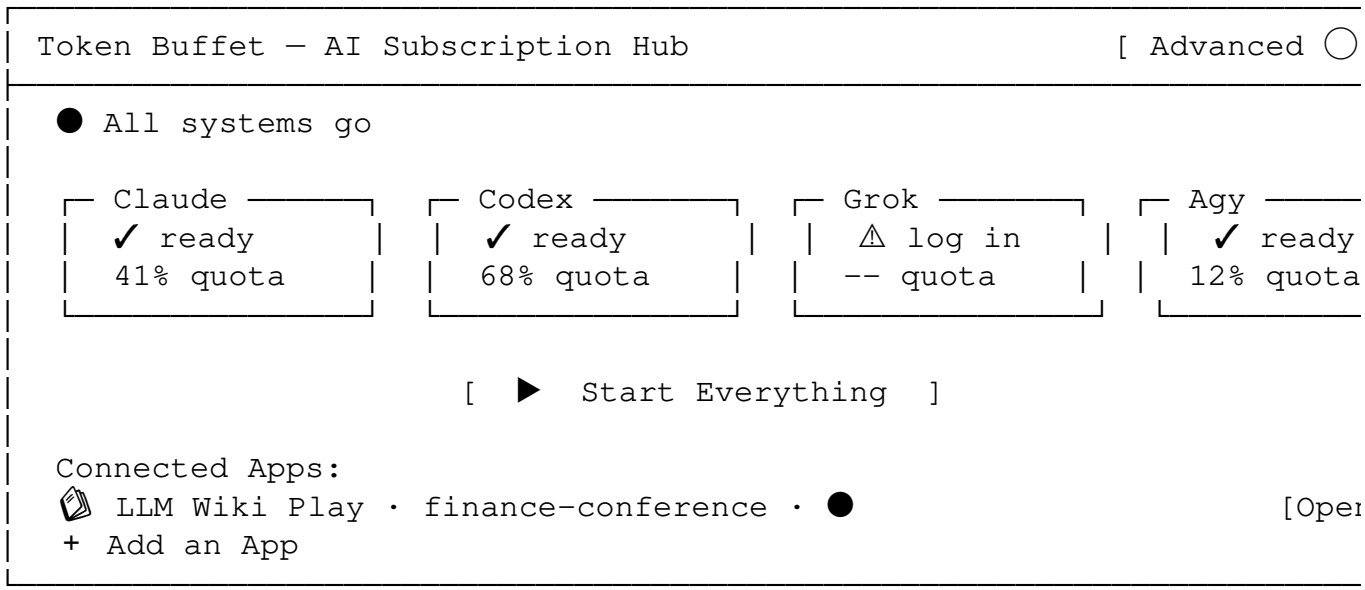
## 1. Core Architectural Position & Design Ethos

The port 8080 dashboard embodies the foundational engineering principles of the Token Buffet ecosystem:

- **Standard Library Only**: Built strictly using Python's standard library (http.server in dashboard/server.py, with static files in dashboard/static/), requiring zero external web frameworks, JavaScript build steps, or heavyweight server dependencies.
- **Config-Driven & Decoupled**: The dashboard contains no hardcoded provider or application logic. Instead, it dynamically renders whatever it discovers on disk by reading directly from filesystem sources of truth:
- **Live Process State & Ports**: Read from data/providers_status/_run_status.json.
- **Quota & Model Capabilities**: Read from data/providers_status/_usage.json and _models.json.
- **Provider Definitions & Settings**: Read from registered_providers//config.json.
- **Router Configuration**: Read from config/router_config.json.
- **App Integrations**: Walked from registered_apps///app_meta.json and app_config.json.
- **Orchestration Over Re-implementation**: Rather than duplicating business logic inside web handlers, the dashboard orchestrates existing operational tools. App registration shells out to integrations/register.py, and re-applying settings invokes the workspace instance's configure.py script.

## 2. Progressive Disclosure UX ("Progressive Hub")

To accommodate both non-technical users and technical operators, the dashboard utilizes a Progressive Disclosure design philosophy. A single header toggle ("Advanced mode") toggles a body.adv CSS class on the page, revealing administrative panels and dense technical data without requiring a page reload or separate build.




Basic Surface (Default View)

- **Designed for everyday monitoring and ease of use**:
- **Hero Status Line**: Displays an immediate visual indicator of overall network health

(e.g., "All systems go" or "All services online").
- **Quota Gauges**: Displays remaining capacity percentages (remaining_pct) and reset countdowns for each provider account, sorted by tightest remaining quota.
- **Service Dots & Hero Actions**: Features visual status dots (🟢 up/ready, 🟡 attention needed, 🔴 down) alongside a one-click "Start everything" button.
- **Connected App Cards**: Displays registered client tools (such as Obsidian vaults or Claudia second-brain workspaces) with direct "Open" launchers.

Advanced Mode (Operator View)

Toggling Advanced Mode reveals granular system details and control surfaces:

- **Process Fleet Control**: Displays exact Process IDs (PIDs), configured ports, and live bound ports for the router (8090), dashboard (8080), and CLI proxies (8000–8003+).
- **Individual Lifecycle Buttons**: Enables launching, killing, or stopping specific proxies, as well as adjusting background usage refresh intervals.
- **Raw Configuration Editors**: Opens inline JSON editors for app_meta.json, app_settings.json, and vault_types.json.
- **Provider Controls**: Exposes tool checkboxes (updating tools.enabled), sandbox environment toggles, model re-scraping, and full model availability tables.

## 3. Key Functional Panels & Subsystems

The dashboard organizes management tasks across these functional panels:

### A. Home (Fleet Monitoring)

Monitors active loopback services through the /api/state payload. It tracks provider status (starting, running, failed, stopped) published in data/providers_status/_run_status.json and surfaces process management controls.

### B. My Apps & Registration Wizard

Manages client application integrations through a step-by-step registration wizard (/api/apps/register):

- **Select App Type**: Choose from available integration types (obsidian, claudia, abject, or a chatbot CLI preset: Claude Code, Codex, Grok, Antigravity).
- **Path & Name**: Browse or enter the local workspace directory (e.g., Obsidian vault path or Claudia root) and assign an instance label.
- **Template/Preset Selection**: Select a preset schema from vault_types.json (such as finance-conference, research-papers, or daily-notes).
- **Execute & Stream**: Spawns integrations/register.py as a subprocess to configuration templates, substitute path placeholders, run initialization scripts, and write native app configuration files.

### C. LLM Providers

Provides an overview of connected vendor CLI subscriptions (Claude Code, Codex CLI, Grok Build, Gemini Antigravity). Displays account quota gauges, model lists from _models.json, and allows operators to toggle supported provider tools (e.g., Bash, Read, Edit, WebSearch). On Windows or macOS, clicking "Log in" triggers an interactive terminal session to handle vendor authentication TUIs.

**D. Router Configuration**

Allows operators to configure router settings stored in config/router_config.json:

- **Routing Mode**: Switch between Usage Mode (load-balancing prompts to the backend with the highest remaining quota percentage) and Custom Mode (following a strict provider preference order).
- **Custom Order**: Drag or reorder the fallback sequence (custom_order) across backends.
- **Tier Mapping**: Inspect and adjust model mappings for the fast, medium, and reasoning semantic tiers.
- **Restart Banner**: Displays a banner noting that routing changes require a router process restart, as active router instances retain a startup snapshot of tier configurations.

**E. Requests & Activity Logs**

Tracks API request history by parsing session logs (logs//proxy/sessions/req_*.log). It correlates request IDs with provider proxies and displays response status without logging or exposing private prompt or completion text.

## 4. Loopback HTTP Control API

The dashboard exposes a set of JSON POST control endpoints and GET state endpoints, listening strictly on 127.0.0.1:8080:

| Endpoint | Method | Operational Purpose |
|---|---|---|
| /api/state | GET | Returns providers, router configuration, registered apps, app types, and fleet status in one payload. |
| /api/router | POST | Reads or rewrites router mode, custom order, and tier configurations in router_config.json. |
| /api/apps/register | POST | Invokes integrations/register.py to scaffold and register a new client app workspace. |
| /api/apps/apply | POST | Re-applies an instance's settings to the client app. |
| /api/providers/ launch / /stop | POST | Launches or terminates individual CLI proxy server processes. |
| /api/providers/ login | POST | Opens an interactive terminal session (via AppleScript on macOS or PowerShell on Windows) for vendor CLI login. |
| /api/providers/ tools | POST | Updates tools.enabled inside registered_providers//config.json. |

## Local Router: Port 8090

In the overall architecture of Token Buffet AI Subscription Hub, the Local Router Gateway operating on port 8090 serves as the central orchestration layer and API entry point. Positioned between client applications and individual provider proxies, the port 8090 gateway abstracts vendor-specific CLI mechanics into a single, standard API endpoint that dynamically distributes prompts across active subscriptions based on performance requirements and quota availability.

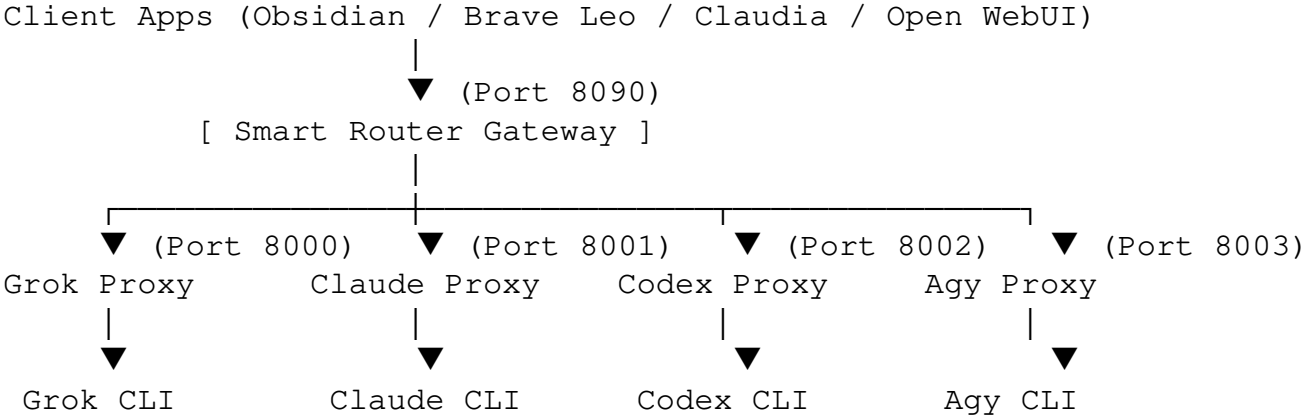
Below is a detailed breakdown of how the Local Router on port 8090 operates within Token Buffet's core architecture.

## 1. Architectural Position & Gateway Role

Token Buffet divides its execution layer into three distinct process types running over local loopback (127.0.0.1):

- **CLI Provider Proxies (Ports 8000–8003+)**: Individual proxy servers spawned per vendor CLI (e.g., Grok on 8000, Claude on 8001, Codex on 8002, Antigravity on 8003).
- **Local Router Gateway (Port 8090)**: The central routing gateway that receives all incoming prompts from client applications and forwards them to eligible provider proxies.
- **Control Center / Dashboard (Port 8080)**: The management interface for monitoring fleet health, managing app registrations, and configuring router settings.

While a direct, standalone launch of the router defaults to port 11434 (matching Ollama's native default port), the standard fleet launcher (tbuffet start or tbuffet start-router) explicitly binds the router gateway to port 8090




## 2. Multi-Format Wire Translation & Universal Endpoint

The router on port 8090 natively translates requests across four standard API wire formats without requiring intermediate proxy gateways:

- **OpenAI Wire Format**: Accepts standard chat completion (/v1/chat/completions), responses (/v1/responses), and model discovery (/v1/models) requests.
- **Anthropic Wire Format**: Accepts /v1/messages requests directly from Anthropic-based applications (such as Claudia or Claude Code) without additional protocol translation.
- **Ollama Wire Format**: Accepts native Ollama API calls (/api/chat, /api/generate, /api/tags).
- **Gemini Wire Format**: Accepts text-only GenerateContent calls (/v1beta/models/{model}:generateContent and :streamGenerateContent).

Because port 8090 answers both OpenAI-compatible and Ollama formats simultaneously, any bring-your-own-model (BYOM) application can connect using a universal configuration pattern:

- **Base Endpoint**: http://127.0.0.1:8090 (Ollama host) or http://127.0.0.1:8090/v1 (OpenAI-compatible).
- **API Key**: Any dummy placeholder string (e.g., ollama).
- **Supported Clients**: Enables immediate routing for desktop interfaces (Open WebUI, Jan, Msty), knowledge tools (Obsidian karpathywiki, Joplin, Logseq, AnythingLLM), browser extensions (Brave Leo), email extensions (Thunderbird Ollama Compose), and system agents (Home Assistant, Raycast, Claudia).

## 3. Semantic Tier Mapping & Model Resolution

Rather than requiring downstream applications to request specific vendor model strings (like claude-3-5-sonnet or grok-4.7), the router on port 8090 exposes three abstract performance tiers:

fast (Quick): Suited for light autocomplete, tag extraction, or ingestion.

medium (Balanced): Suited for standard chat, drafting, and daily note synthesis.

reasoning (Best): Suited for complex reasoning, multi-document analysis, and deep synthesis.

The router resolves these generic tiers to concrete provider models using exact model IDs frozen per provider in config/router_config.json. Those IDs are populated once from each provider's tier regex (defined in registered_providers//config.json), matched against the models returned by that running proxy's /models endpoint. When a request hits port 8090, the router resolves the destination by tier, exact model ID, provider name, or fuzzy match, falling back to the medium tier (or returning an HTTP 503 if unresolvable).

## 4. Smart Load Balancing & Quota-Aware Routing

The port 8090 router manages traffic across connected CLI proxies using two configurable load-balancing modes:

- **Usage Mode**: Polls each running proxy's GET /usage endpoint to identify remaining capacity. It dynamically routes incoming prompts to whichever connected provider currently holds the highest remaining usage quota percentage.
- **Custom Mode**: Bypasses quota sorting to follow a hardcoded provider preference order (custom_order), such as prioritizing Claude before falling back to Codex, Antigravity, or Grok.

To ensure zero latency overhead during active request routing, the router never invokes TUI scrapers inline. Instead, it polls each running proxy's endpoints at startup and on a configurable background schedule (defaulting to 1,800 seconds / 30 minutes).

## 5. Fault Tolerance, Retries, and Buffering

To maintain connection stability across local CLI backends, the router enforces strict failover mechanisms:

- **Backend Skipping**: Automatically skips known depleted, stopped, or offline proxies.
- **Pre-Stream Failover**: If a proxy encounters a connection failure, rate limit (HTTP 429), or CLI execution error before streaming starts, the router advances to the next candidate provider within the tier.
- **Disk Buffering**: Non-streaming requests are buffered in memory and spilled to disk if payload sizes exceed 1 MiB, allowing failed attempts to be retried cleanly without corrupting data.
- **Streaming Prelude Handling**: For streaming requests ("stream": true), the router holds only the initial response prelude until the first text chunk arrives. Failures before text generation trigger a retry on an alternate backend, whereas errors occurring mid-stream are passed directly to the client as stream errors because partial text is already visible.

## 6. Client Tool Handoff Mechanics

When client applications submit prompts containing function calls or tool declarations, the router delegates execution cleanly:

The router passes the request to the vendor CLI with execution flags (such as --tools "") to request a structured tool-call decision from the model.

The vendor CLI returns the tool decision to the router on port 8090, which delivers it back to the client application for local execution.

## Proxy Runtime: Standard HTTP server

In the larger architecture of the Token Buffet AI Subscription Hub, the Proxy Runtime serves as the dedicated, provider-facing execution layer. Each connected command-line interface (CLI) subscription—such as Claude Code, Codex CLI, Grok Build, and Gemini Antigravity—is managed by an individual, lightweight HTTP proxy process.

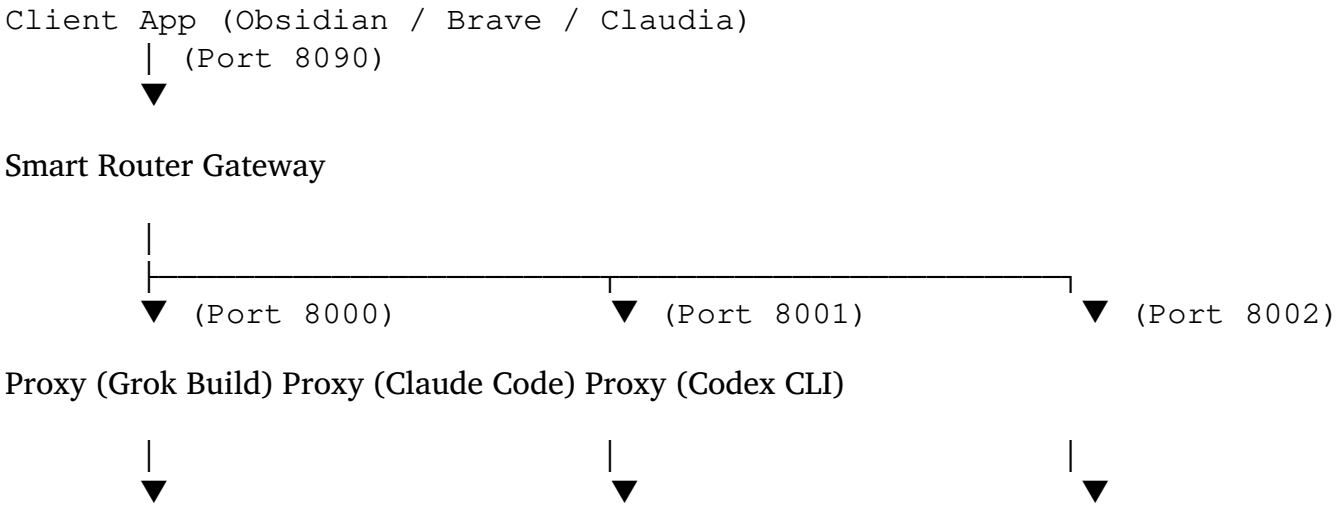
Built strictly around Python's standard library http.server, the proxy runtime embodies Token Buffet's core design philosophy: a minimal, zero-dependency runtime that avoids heavy web frameworks or external build steps.

Below is an in-depth discussion of the Proxy Runtime's design, operational responsibilities, and position within the overarching system architecture.

## 1. Position Within the Three-Pillar Architecture

Token Buffet decouples local AI bridging into three primary process layers running on dedicated loopback ports:

- **Smart Router Gateway (Port 8090)**: Accepts incoming client API calls matching OpenAI (/v1), Anthropic (/v1), and Ollama (/api) wire formats.
- **Control Center Dashboard (Port 8080)**: Provides a single-page management console (also built on http.server) to monitor fleet health, manage app registrations, and control process lifecycles.
- **CLI Provider Proxies (Ports 8000–8003+)**: Individual proxy server processes (cli_proxy_server) spawned per vendor CLI. Each proxy binds to a configured loopback port (e.g., Grok on 8000, Claude on 8001, Codex on 8002, Antigravity on 8003) and handles direct communication with that specific vendor's interactive CLI.



## 2. Core Responsibilities of the Proxy Runtime

The proxy server process abstracts interactive vendor CLIs into standard HTTP endpoints while managing local process isolation and health monitoring:

### A. Request & Stream Proxying

- **Wire Translation**: Each proxy converts incoming HTTP requests from the Smart Router into CLI commands and streams the output back to the router.
- **Error Handling & Spooling**: For non-streaming requests, proxy execution output is buffered (and spooled to disk if exceeding 1 MiB). If a non-streaming CLI execution fails, the proxy returns an HTTP 502 Bad Gateway error; for streaming requests, it emits a structured error event.
- **Tool Handoff**: When client requests contain tool declarations, proxies invoke the vendor CLI with specific tool flags (e.g., --tools "") to obtain a structured tool-call decision without executing the tool locally, delegating actual execution back to the client app.

### B. Decoupled Extraction & Status Caching

Rather than forcing the Smart Router to scrape vendor CLIs during active API requests, each proxy independently handles resource extraction:

- **Background Extraction**: Proxies spawn pseudo-terminal (PTY) or subcommand tasks to run vendor status commands (such as /usage and /models) inside the interactive CLI TUIs.
- **Asynchronous JSON Export**: Extracted status metrics are written to structured JSON files in data/providers_status/:

_usage.json: Tracks canonical remaining quota percentages (remaining_pct) and reset timestamps.

_models.json: Lists supported underlying provider model IDs.

_run_status.json: Contains the live operational status and bound port published by the proxy.

- **Cached Polling**: Each proxy binds its port first, then reads usage and models in a background thread and refreshes them every --refresh-interval seconds (default 1,800 seconds / 30 minutes). The Smart Router polls the proxy's /usage and /models endpoints instead of reading the status files, avoiding any latency overhead during prompt routing.

### C. Pre-flight Health Validation & Life-Cycle States

- **Startup Read**: After binding its port, a proxy reads usage and models in a background thread. If the CLI displays an updater, login prompt, or unparseable screen, the read fails, the proxy records the captured reason as usage state unavailable, and it retries every 300 seconds.
- **Run Status Reporting**: A proxy advertises its status as starting, running, failed, or stopped in _run_status.json.
- **Router Protection**: The Smart Router only directs traffic to proxies that are actively

running and listening on their assigned ports. Stale status files from forced shutdowns or failed startups are automatically ignored.

## 3. Parallel Execution & Sandbox Home Isolation

Native vendor CLIs frequently enforce global user configuration paths and SQLite database file locks, which normally prevent running multiple concurrent sessions.

To overcome this, the Proxy Runtime isolates each proxy process using dedicated sandbox directories:

During execution, the proxy sets isolated session environment variables—such as GROK_HOME for Grok, CODEX_HOME for Codex, or data/provider_homes/agy for Antigravity.

This sandbox isolation allows multiple downstream applications (e.g., Obsidian vaults, Claudia assistants, and browser plugins) to execute prompts in parallel without encountering database lock errors or overwriting user settings.

## 4. Fleet Control & Cross-Platform Management

The Proxy Runtime integrates directly with Token Buffet's fleet management and dashboard tools:

- **Control Center Operations**: From the dashboard at http://127.0.0.1:8080, operators can launch individual proxies, kill process IDs (PIDs), trigger terminal logins, or manage enabled tool sets via POST API endpoints (/api/providers/launch, /api/providers/stop, /api/providers/login).
- **Cross-Platform Adaptability**: On macOS (sys.platform == "darwin"), process lifecycle management is delegated to an isolated platform adapter (src/tokenbuffet/platforms/macos/processes.py) that uses psutil to inspect and terminate process trees reliably without relying on Linux /proc assumptions. On Windows, process tracking falls back to native command-line process scans.

## Extractor Subsystem: Usage and model capture

In the architecture of the Token Buffet AI Subscription Hub, the Extractor Subsystem (located in src/tokenbuffet/extractors/) serves as the automated monitoring and state-capture engine. Its purpose is to periodically capture resource quotas, reset timestamps, and model capabilities from interactive command-line interface (CLI) tools—such as Claude Code, Codex CLI, Grok Build, and Gemini Antigravity—without introducing latency into live API requests.

By decoupling TUI status extraction from prompt handling, the subsystem provides the structured operational data required by both the Smart Router Gateway (Port 8090) for load balancing and the Control Center Dashboard (Port 8080) for visual fleet monitoring.

## 1. Capture Mechanics (PTY & Subcommands)

Because vendor CLIs do not offer standard developer status APIs, the Extractor Subsystem relies on background automation to interact directly with vendor interfaces:

- **Pseudo-Terminal (PTY) Sessions & Subcommands**: The engine spawns background pseudo-terminal tasks (using PTY/pexpect dependencies) or subcommands to issue

vendor-specific status commands (such as /usage in TUI screens or grok models / / models subcommands).
- **Background Usage Read**: Each proxy binds its port first, then reads its own usage and models in a background thread, retrying every 300 seconds on failure. If the vendor CLI displays an unexpected screen such as an updater, missing login, or unparseable interface, the read fails, the proxy records the captured reason as usage state unavailable, and the last valid usage JSON stays on disk.

## 2. Decoupled State Export (data/providers_status/)

Rather than performing live screen scraping when an API request hits the gateway, the Extractor Subsystem writes all captured status metrics asynchronously to structured JSON files inside data/providers_status/:

_usage.json:

Captures plan details (plan), timestamp (gathered_at), and period-based quota limits (usage.limits.).

Tracks used_pct, the canonical remaining_pct (remaining quota percentage), and reset timestamps (resets, e.g., "October 15, 18:30").

_models.json:

Stores supported model definitions under models.available[{id, name}] alongside the default provider model.

Serves as the single source of truth for populated model dropdowns, model discovery endpoints (/v1/models, /api/tags), and tier resolutions.

_run_status.json:

Published directly by each running proxy to advertise its bound loopback port (e.g., { "port": 8000 }) and process lifecycle state (starting, running, failed, or stopped).

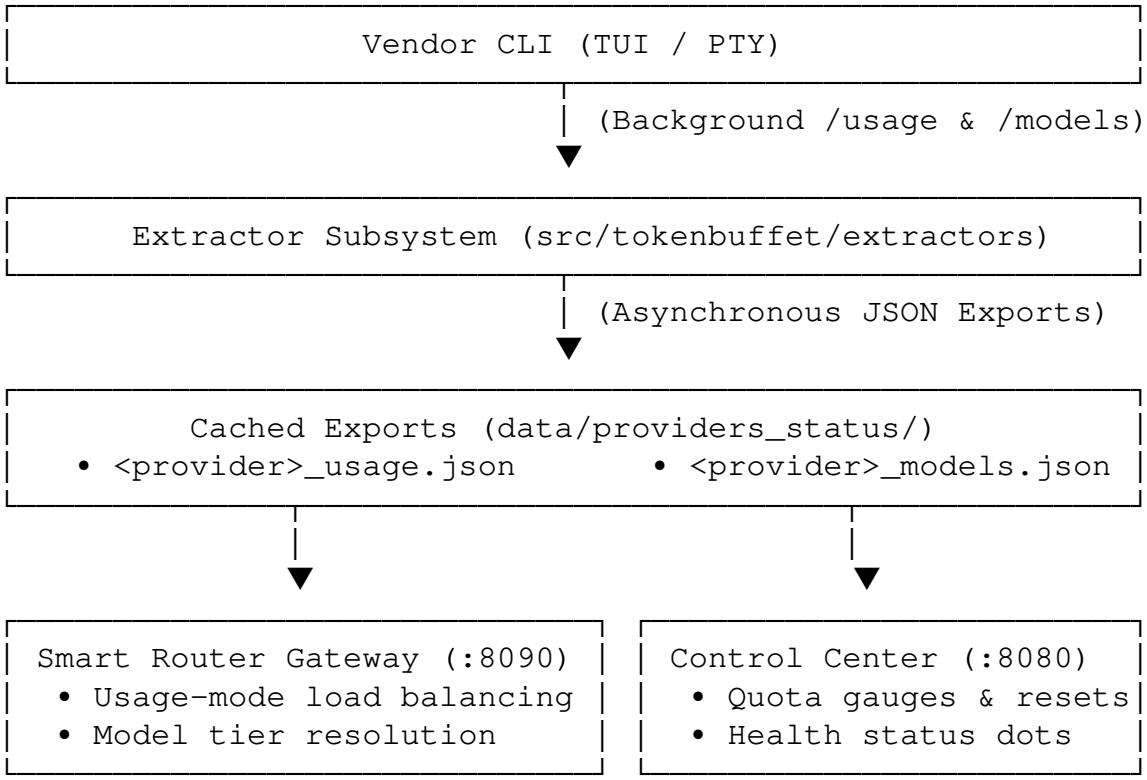
## 3. Caching & Asynchronous Polling Strategy

To ensure zero latency impact on live user requests, the router and dashboard never invoke TUI scrapers directly during prompt execution:

- **Cached Proxy Endpoints**: Individual proxy servers expose GET /usage and GET / models HTTP endpoints. These endpoints return cached JSON data when fresh and re-trigger background extraction only when files are missing or stale.
- **Configurable Refresh Lifetime**: The default cache lifetime is set to 1,800 seconds (30 minutes), adjustable via the --refresh-interval CLI option.
- **Router Background Schedule**: The Smart Router on port 8090 polls each active proxy's /usage and /models endpoints at startup and periodically on its own background refresh schedule. The router never reads raw status files or executes bulk scraping scripts inline.

## 4. Strategic Role in Core Architecture

The Extractor Subsystem acts as the data foundation for the rest of Token Buffet's core architecture:




- **Enabling Smart Load Balancing**: By providing canonical remaining_pct metrics across all active subscription accounts, the subsystem powers the Smart Router's Usage Mode. The router sorts eligible backends by remaining headroom and routes incoming traffic to whichever subscription currently has the highest available capacity.
- **Driving the Control Center Dashboard**: The single-page dashboard at port 8080 reads directly from data/providers_status/ to render visual quota gauges, reset countdowns, service status dots (green/amber/red), and model availability tables. This keeps the operator dashboard responsive and decoupled from CLI execution details.

# Monitoring & Dashboard

The Control Center Dashboard at `http://127.0.0.1:8080` shows what each subscription has left, whether each service is running, and what requests have recently passed through the router. It reads the files the proxies and extractor export, and never shows prompt or response text.

## 1. Usage Gauges

Home shows the remaining quota percentage and reset time per provider period, sorted by each provider's tightest remaining quota. A missing export is shown as unknown. The figures come from `usage.limits.<period>.remaining_pct` in each `<provider>_usage.json`.

## 2. Service Status

Each proxy publishes its bound port and state (`starting`, `running`, `failed`, `stopped`) to `<provider>_run_status.json`, along with a usage state of `reading`, `ready`, or `unavailable`. The dashboard turns these into status dots and a hero line such as "All systems go", with a one-click "Start everything" button.

## 3. Activity Feed

The Activity tab parses per-provider session logs at `logs/<provider>/proxy/sessions/req_*.log` and correlates request ids with router records. It shows timestamps and outcomes, and marks the app as unknown when the client sends no identifying header. Full payloads are written only when Debug logging is on, and are not shown in the page.

## 4. Advanced Mode

A single header switch reveals process ids, live ports, launch, stop and restart buttons, provider tool checkboxes (`tools.enabled`), the Router tab with mode, `custom_order`, and tier model ids, and the Activity tab. Provider tool changes need a proxy restart, and router changes need a router restart.

## 5. Extractor Subsystem

Usage and model data come from the extractor, which captures each vendor CLI's TUI screens and parses them with configured regexes, as described under Core Architecture.

## Service Status: Live port and process state

In the architecture of the Token Buffet AI Subscription Hub, Service Status provides real-time operational visibility into active process states, bound network ports, and overall system health across the bridge.

Because Token Buffet operates as a multi-process bridge—coordinating the Smart Router Gateway (Port 8090), the Control Center Dashboard (Port 8080), and individual LLM Provider CLI Proxies (Ports 8000–8003+)—monitoring live port bindings and process states is essential for reliable prompt routing and non-disruptive process management.

Below is an in-depth breakdown of what the sources specify regarding live port tracking, process state lifecycle management, router integration, and dashboard visualization.

## 1. Mechanism of Live Port & Process Capture (_run_status.json)

To adhere to Token Buffet's decoupled, config-driven design ethos, the Control Center Dashboard does not hardcode proxy port numbers or execute intrusive screen-scraping scripts to check service health. Instead, it relies on lightweight, filesystem-based status exports:

- **Decoupled Run-Status File**: Whenever an individual CLI proxy process initializes, it publishes its live operational state to disk at data/providers_status/_run_status.json.
- **Live Port Binding**: The file contains a JSON payload specifying the exact loopback port bound by the proxy (e.g., {"port": 8000}).
- **Lifecycle States**: The system tracks four explicit lifecycle states for each proxy component:

starting: The proxy process has spawned and is binding its port.

running: The proxy is actively listening on its assigned port.

failed: The proxy failed to start, for example because of a port conflict.

stopped: The proxy process was cleanly terminated or killed by an operator or system signal.

- **Process ID (PID) Tracking**: The dashboard tracks system Process IDs (PIDs) alongside run-status files to allow targeted process management (such as launching, stopping, or killing specific proxies).

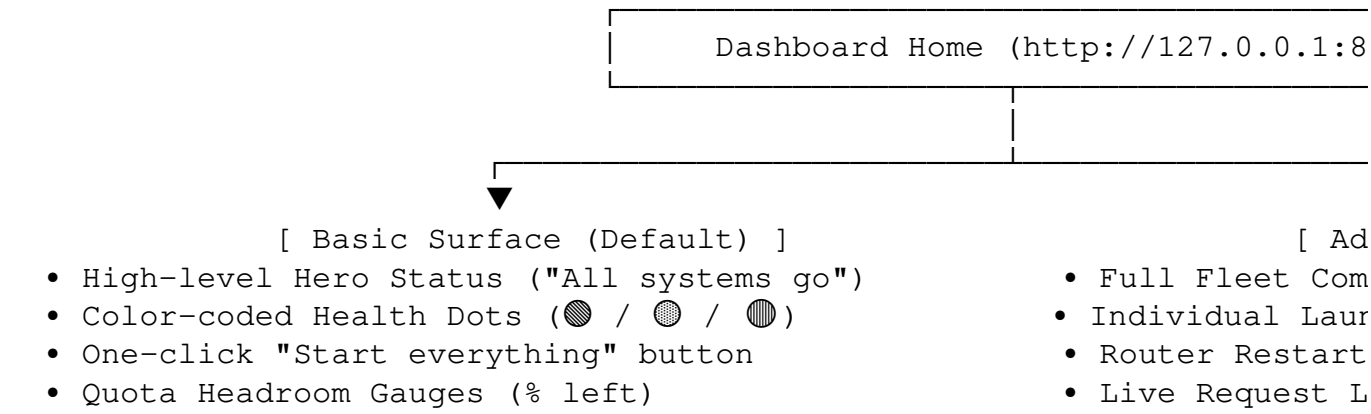
## 2. Integration with the Smart Router (Fault Tolerance & Skipping)

Live port and process status tracking directly drives the fault-tolerance logic of the Smart Router Gateway (Port 8090):

- **Active Listener Rule**: The router enforces a strict rule: it only dispatches client prompts to a provider that is explicitly marked as running AND whose bound port is actively listening on the loopback interface.
- **Startup Read**: After binding its port, each proxy reads its usage in a background thread. If a vendor CLI displays an unexpected screen (e.g., a mandatory CLI update prompt) without returning quota metrics, the read fails and the proxy logs the captured reason and reports usage state unavailable.
- **Automatic Bypassing**: When a provider status is marked as failed or stopped, the Smart Router automatically skips that provider during model selection in both Usage Mode and Custom Mode.
- **Stale File Protection**: If a proxy process is forcibly terminated (leaving behind a stale _run_status.json file on disk), the router checks whether the port is actively listening before attempting a connection, preventing client requests from hanging on dead sockets.
- **Fleet Startup Sequence**: The tbuffet start fleet launcher waits for all proxy startups to complete before launching the router, ensuring the router's initial model snapshot accurately reflects all currently running providers.

## 3. Dashboard Visualization: Basic vs. Advanced Mode

The Control Center Dashboard (http://127.0.0.1:8080) translates backend process states and port bindings into visual status displays using a Progressive Disclosure user experience:




- **Basic Surface (Default View)**:
- **Visual Status Dots**: Displays component readiness using universal color-coded indicators: 🟢 Green (up / ready / connected), 🟡 Amber (needs attention / login required / low quota), 🔴 Red (down / error / failed), and ○ Grey (disabled / unknown).
- **Hero Summary**: Displays top-level status lines such as "● All systems go" or "✓ All services online (providers + gateway)" alongside a single "Start everything" button.

Advanced Mode (body.adv):

Toggling the "Advanced mode" header switch morphs the interface into an operator console that exposes complete fleet process details.

- **Fleet Table**: Displays a structured matrix showing every component (router, dashboard, claude, codex, grok, agy), its configured port (config.server.port), its live bound port from _run_status.json, its active PID, and its current state.
- **Granular Process Controls**: Provides individual Launch (spawns proxy on default port), Kill (terminates PID), Login (opens terminal authentication), and Restart Router buttons.

## 4. Cross-Platform Process & Terminal Management

Because Token Buffet operates across both Windows and macOS environments, process state management adapts to the underlying host operating system:

- **Windows Process Management**: Uses native Windows process commands and the tbuffet start, start-provider, and start-router commands to spawn, monitor, and terminate process trees.

macOS Process Isolation (src/tokenbuffet/platforms/macos/):

processes.py: Uses the psutil library to verify service command lines and cleanly stop process trees, avoiding Linux /proc assumptions that fail on macOS.

terminal.py: Uses AppleScript to launch interactive provider login sessions and chatbot workspaces inside visible Apple Terminal windows, passing environment variable overrides while tracking process IDs cleanly.

- **HTTP API Fleet Control**: The dashboard exposes endpoint controls (GET /api/state, POST /api/providers/launch, POST /api/providers/stop, POST /api/providers/login) that handle process lifecycle events and return structured success or failure toasts back to the web interface.

## Usage Gauges: Remaining quota percentages

In the Token Buffet AI Subscription Hub, Usage Gauges represent the visual and data-driven mechanisms on the Control Center Dashboard (http://127.0.0.1:8080) that display the remaining usage capacity (remaining_pct) and reset countdowns across all connected CLI subscription accounts.

In the broader context of Monitoring & Dashboard design, Usage Gauges serve as the critical bridge between raw CLI status extraction and intelligent prompt dispatching across vendor subscriptions.

## 1. Visual Representation & User Interface

On the Dashboard's Home surface ("See what you have left") and the LLM Providers panel, remaining account capacity is rendered as horizontal percentage bars rather than raw token counts or unparsed CLI logs:

- **Headroom Percentages**: Displays the canonical remaining quota percentage (e.g., 99.87%, 84%, 68%, or 12%) for each provider period (such as 5-hour, daily, or weekly limits).
- **Reset Timestamps & Countdowns**: Shows clear reset schedules (e.g., "resets Oct 27,

10:59am" or "Resets Refreshes in 36m") so operators know exactly when exhausted quotas will replenish.
- **Triage & Status Indicators**: Depleted quotas (remaining_pct < = 0), missing status exports (N/A), or unauthenticated CLI sessions trigger amber/red status indicators (such as ⚠ Quota depleted or ⚠ Not logged in), making account issues unmissable.
- **Headroom Sorting**: Gauges on the Home view are dynamically sorted by each account's tightest remaining capacity, giving operators an instant glance at which accounts have the most room.

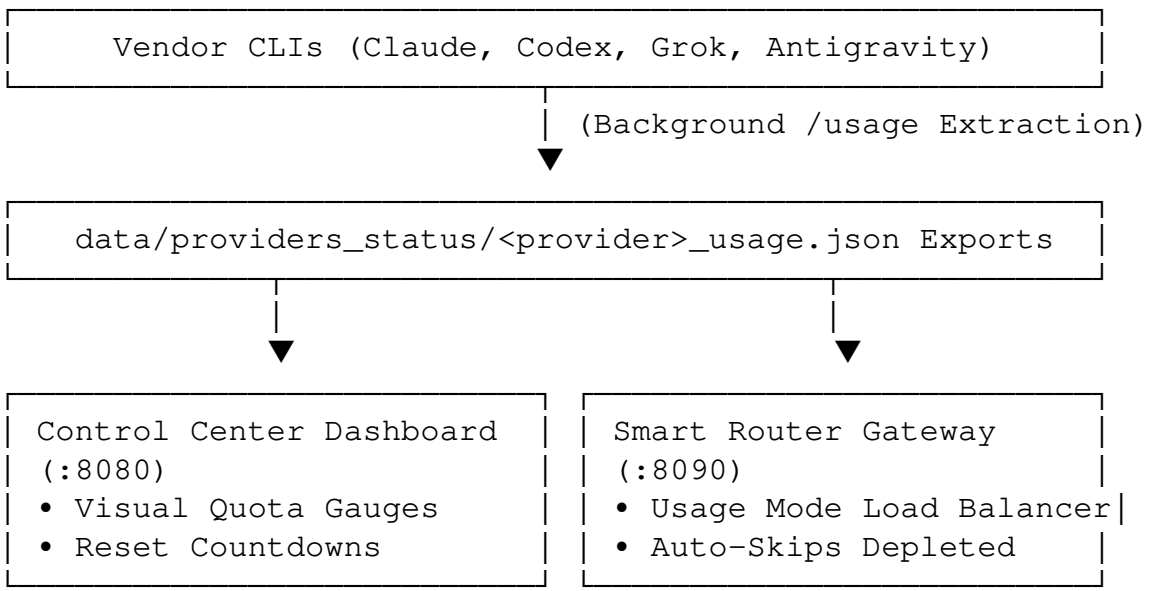
## 2. Under the Hood: The Extractor Subsystem & Caching

Usage Gauges do not execute slow, inline screen-scraping when a user opens the dashboard or submits an API request:

- **Asynchronous Extraction**: The Extractor Subsystem spawns background pseudo-terminal (PTY) tasks or subcommands to issue status commands (such as /usage) directly inside interactive vendor CLIs.
- **Structured Status Exports**: Extracted metrics are saved asynchronously to disk as JSON status files at data/providers_status/_usage.json.
- **Canonical Schema**: Within the status JSON, the metric usage.limits..remaining_pct serves as the single source of truth for quota calculations, with used_pct stored as its complement.
- **Cached Polling**: Individual proxies serve their usage status via GET /usage endpoints with a default cache lifetime of 1,800 seconds (30 minutes). The router polls these endpoints on a schedule, while the dashboard reads the exported status files, ensuring live prompt execution remains fast and responsive.

## 3. Strategic Role: Driving Smart Routing & Usage Mode

Beyond providing visual monitoring for operators, the underlying data powering the Usage Gauges directly drives the Smart Router Gateway (Port 8090):




- **Usage Mode Load Balancing**: When downstream applications (such as Obsidian, Brave Leo, Raycast, or Claudia) send prompts to port 8090 using abstract semantic performance tiers (fast, medium, reasoning), the router operates in Usage Mode by reading these exact remaining_pct metrics.

- **Capacity Maximization**: Prompts are automatically dispatched to whichever connected provider currently holds the highest remaining quota headroom, spreading workload across subscriptions without paying per-token API bills.
- **Automatic Depletion Skipping**: Providers whose Usage Gauges report remaining_pct $<= 0$ or offline states are automatically bypassed in favor of alternative backends supporting the requested tier.

## Advanced Mode: Provider tools and router config

In the architecture of the Token Buffet AI Subscription Hub, Advanced Mode serves as the opt-in operator surface within the Control Center Dashboard (http://127.0.0.1:8080).

Adhering to a Progressive Disclosure UX design ("Progressive Hub"), the dashboard remains simple and uncluttered by default for everyday users, displaying hero status lines, visual quota gauges, and one-click app launchers. Toggling the "Advanced mode" switch in the header adds the body.adv CSS class, revealing deep technical controls, raw configuration editors, process details, Provider Tools, and complete Router Configuration panels.

## 1. Provider Tools in Advanced Mode

On the LLM Providers / Accounts panel, Advanced Mode grants operators direct control over which client capabilities and agentic tool integrations are exposed to each backend CLI.

- **Target File & Writable Field**: For each discovered provider (Claude, Codex, Grok, Gemini Antigravity), Advanced Mode renders a card displaying its model catalog and available tools. Operators manage tool access via interactive checkboxes (such as Bash, Edit, Read, WebSearch). Checking or unchecking these tools writes directly to tools.enabled inside registered_providers//config.json. Notably, tools.enabled is the only field in the provider definition file that the dashboard ever modifies on disk.
- **Proxy Restart Requirement**: Because running proxy processes hold CLI execution settings in memory, any edits made to provider tools require a proxy restart to take effect.
- **Tool Handoff Synergy**: Provider definitions can also specify execution.tool_handoff_args (such as passing --tools "" for Claude Code and Grok Build). This instructs the backend CLI engine to generate a structured tool-choice decision while leaving actual tool execution safely in the hands of the downstream client workspace.

## 2. Router Configuration in Advanced Mode

While the basic surface collapses routing strategy into a simple plain-language choice ("Spread work across accounts" vs. "Always use my best account first"), Advanced Mode unveils the full Routing panel for precise control over prompt dispatching.

The Routing panel reads and edits config/router_config.json across three core dimensions:

- **Routing Mode (mode)**:
- **Usage Mode ("usage")**: Operates as a load-balancing strategy that uses the remaining_pct metrics the router polls from each running proxy's /usage endpoint and automatically routes prompts to whichever active backend currently holds the highest available quota headroom.
- **Custom Mode ("custom")**: Bypasses quota equalization and directs prompts strictly according to a hardcoded provider preference sequence.

- **Preferred Account Order (custom_order)**:

Operators can drag or reorder an interactive provider priority list (e.g., ["claude", "codex", "agy", "grok"]). In Custom Mode, the gateway evaluates candidate backends top-to-bottom, advancing to the next provider only if a higher-ranked backend is offline, depleted (remaining_pct $<=$ 0), or rate-limited.

- **Semantic Tier Pinning (tiers)**:

Displays and allows customization of the frozen mapping that translates abstract client performance tiers (fast / Quick, medium / Balanced, reasoning / Best) into exact vendor model IDs (e.g., mapping reasoning to gemini-3.1-pro-high, Opus, GPT-6-Astra, or grok-4.7).

The Startup Snapshot & Restart Rule

When router settings are saved, config/router_config.json is updated on disk immediately. However, running router instances retain a startup snapshot of configuration state to prevent mid-session routing drift. The dashboard explicitly flags this with an in-situ banner ("Changes apply after a router restart") and provides a one-click "Restart router" button to apply updates instantly.

## 3. Summary: Basic Surface vs. Advanced Mode Controls

| Dashboard Feature | Basic Surface (Everyday User) | Advanced Mode (Operator / Tinkerer) |
|---|---|---|
| UX Target | Calm, card-based overview; zero technical jargon. | Dense body.adv table view exposing ports, PIDs, and logs. |
| Provider Management | Status dots (🟢/🟡/🔴), quota gauges, and "Log in" buttons. | Sandbox toggles, model re-scraping, and Tools checkboxes (tools.enabled). |
| Router Controls | Hidden or shown as a single plain toggle (Spread work vs. Use best first). | Full custom_order drag list, tier-to-model pinning, and router restart triggers. |
| App Management | Add-an-App 3-step wizard and one-click "Re-apply" buttons. | Raw JSON editors for app_meta.json, app_settings.json, and vault_types.json. |

## Activity Feed: Session request logs

In the architecture of the Token Buffet AI Subscription Hub, the Activity Feed (Session Request Logs) serves as the operational audit log and request-tracing surface within the Control Center Dashboard (http://127.0.0.1:8080).

While Usage Gauges track account capacities and Service Status monitors process health, the Activity Feed tracks individual AI prompts as they flow through the system. It allows operators to trace request lifecycles across client apps, the Smart Router gateway, and backend CLI proxies without compromising user privacy.

## 1. Data Source & File Correlation Mechanics

The Activity Feed does not rely on active database connections or external log aggregators. Adhering to Token Buffet's file-based, config-driven ethos, it dynamically parses per-provider session logs stored on disk:

- **Session Log Format**: Reads log files located in logs//proxy/sessions/req_*.log.
- **Encoded Filename Metadata**: Log filenames encode metadata directly into their title structure:

req_.log

For example, req_claude_20261011_204213_6b959df8.log identifies the target provider, precise timestamp, and unique request ID.

- **Status Inference via Body Markers**: To determine request outcomes without heavy parsing, the dashboard inspects body markers inside the session log:
- **Output received**: → Completed / OK
- Proxy Error → Error
- Absence of termination markers → In Progress
- **Console Summaries**: Both router and CLI proxy consoles emit synchronized REQUEST, FAIL, and DONE summary lines sharing the exact same request ID. Provider error messages are whitespace-normalized and capped at 500 characters to prevent console spam.

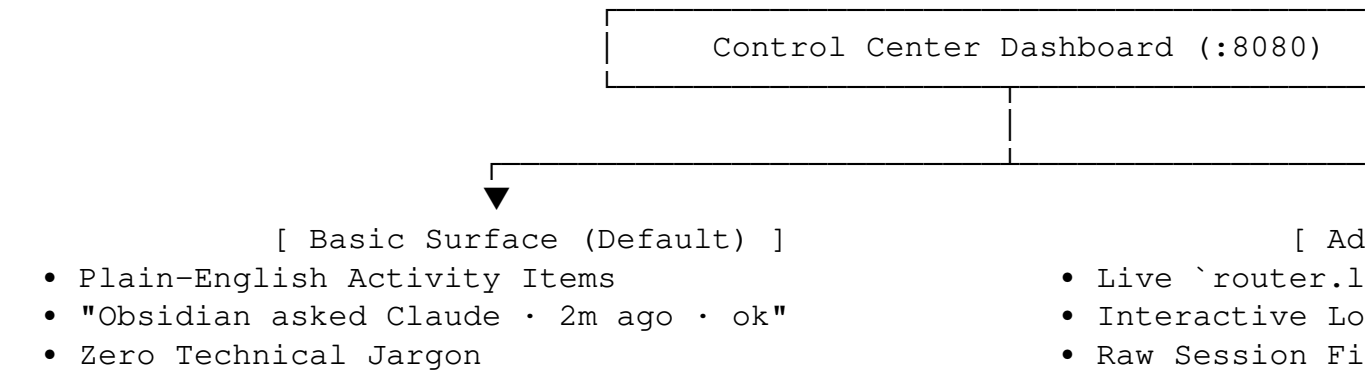
## 2. Privacy-First Logging Architecture

A core design principle of Token Buffet's Activity Feed is strict prompt and response privacy:

- **Zero Content Exposure**: By default, the Activity page displays no prompt text or response body content. Operators can see that a request occurred, which provider handled it, and whether it succeeded, but cannot read the user's private text.
- **Debug Payload Isolation**: Full request/response payloads (containing system prompts, user inputs, and model outputs) are recorded only when Debug logging is explicitly enabled. These raw JSON payloads are written to isolated files at logs//proxy/raw_requests/req_.json for local debugging and are never rendered on the web dashboard surface.

## 3. Progressive Disclosure: Basic vs. Advanced Activity Surfaces

The Activity Feed adapts its presentation based on the dashboard's Progressive Disclosure UX model (Progressive Hub):




Basic Surface (Everyday User): Translates request logs into plain-langu

Advanced Mode (body.adv): Turning on the "Advanced mode" header toggle unlocks the full Activity operator tab:

- **Live Log Tailing**: Enables live tailing of router.log stream lines.

- **Filtering & File Opening**: Allows operators to filter requests by provider or status and open raw log files directly from the interface.

### 4. Client Attribution & Unknown App Headers

When downstream software makes a request to port 8090, the router attempts to identify which client application initiated the call:

- **App Headers**: If a client application supplies an identifying user-agent or client header, the Activity Feed labels the request with the application name (e.g., Obsidian, Brave Leo, Claudia).
- **Handling Unidentified Clients**: If a downstream client does not provide identification headers, the dashboard marks the app attribution as "unknown" rather than guessing or mislabeling the traffic source.

## Smart Routing & Tiers

The Smart Router (port 8090) decides which subscription answers each request. Routing happens in two steps: resolve the requested model to a tier and its candidate backends, then choose among the backends by routing mode.

### 1. Semantic Tiers

- **fast (Quick)**: Light autocomplete, tag extraction, and ingestion.
- **medium (Balanced)**: Standard chat, drafting, and daily note synthesis.
- **reasoning (Best)**: Complex reasoning, multi-document analysis, and deep synthesis.

Each provider's `config.json` holds a regex per tier. The first time a running proxy's models are readable, the router matches the regexes and freezes the newest matching exact model ids into `config/router_config.json`, with a default effort per tier. Hand edits to that file win.

### 2. Resolution Order

A request is resolved by tier, then exact model id, then provider name, then fuzzy match, and finally falls back to the medium tier. If no medium fallback exists the router returns a structured HTTP 503.

### 3. Routing Modes

- **Usage mode**: Picks the backend with the most remaining quota, skipping depleted (`remaining_pct <= 0`), stopped, or offline backends.
- **Custom mode**: Follows the global `custom_order` preference list, for example claude, codex, agy, grok.

### 4. Failover and Tool Handoff

Provider failures, connection failures, HTTP 429, and explicit error events before streaming advance to the next candidate. When a request carries client tools, the router asks a backend CLI for one structured tool-call decision and returns it to the client, which executes the tool itself. Claude and Grok configs pass `--tools ""` for this.

## 5. Operator Control

The dashboard's Router tab edits the mode, `custom_order`, and tier model ids. A running router keeps its startup snapshot, so saved changes apply after a router restart.

## Semantic Tiers: Fast, medium, and reasoning

In the architecture of the Token Buffet AI Subscription Hub, Semantic Tiers (fast, medium, reasoning) form the foundational abstraction layer used by the Smart Router Gateway (Port 8090).

Because flat-rate command-line interface (CLI) subscriptions—such as Claude Code, Codex CLI, Grok Build, and Gemini Antigravity—do not issue traditional per-token developer API keys, downstream software applications are decoupled from specific vendor model strings (like claude-3-5-sonnet or grok-4.7). Instead, client applications issue requests using generic performance tiers. The router translates these requests into concrete vendor models and dynamically balances the workload across connected subscriptions based on available quota headroom.

Below is an in-depth breakdown of what the sources specify regarding the definition, resolution, load balancing, per-task assignment, and operator management of semantic tiers.

## 1. The Three Semantic Model Tiers

The router exposes three primary performance tiers, which correspond to clear plain-language labels across the system's user interfaces:

fast (Plain Language: "Quick"): Tailored for high-frequency, low-latency, lightweight background tasks such as text autocomplete, document ingestion, and tag extraction.

medium (Plain Language: "Balanced"): The default baseline tier suited for general chat sessions, standard drafting, daily note synthesis, and routine summarization.

reasoning (Plain Language: "Best"): Reserved for heavy compute operations, multi-document analysis, periodic vault linting, deep knowledge synthesis, and complex problem-solving.

## 2. Tier Mapping & Resolution Mechanics

The router resolves incoming tier requests to concrete vendor CLI models using a multi-stage configuration structure:

- **Provider Regular Expressions**: Each provider configuration (registered_providers//config.json) defines regex patterns under its tiers object that match its supported CLI models (e.g., mapping fast to build-fast$, medium to grok-4\.6, and reasoning to grok-4\.7$).

Centralized Router Map (router_config.json): The router matches these regexes once against each running proxy's /models endpoint and freezes the newest matching exact model IDs per provider in config/router_config.json. For example, the reasoning tier maps across connected backends to specific model IDs such as Gemini Antigravity (gemini-3.1-pro-high), Claude (Opus), Codex (GPT-6-Astra), and Grok (grok-4.7).

- **Strict Resolution Sequence**: When an incoming API request reaches port 8090, the router attempts model resolution in the following sequence:

Exact Semantic Tier (fast, medium, or reasoning)

Exact Model ID Match

Provider Name Match

Fuzzy Model Match

Fallback to medium Tier

If a request cannot be mapped and no medium fallback exists, the router returns a structured HTTP 503 error.

## 3. Integration with Smart Load-Balancing Modes

Once a tier is selected, the router chooses which specific backend provider handle the prompt based on the active routing mode:

- **Usage Mode (mode**: "usage"):

The router uses the remaining-quota figures it polls from each running proxy's /usage endpoint, which the proxies obtain asynchronously from the background extractor subsystem.

It ranks all running backends supporting the requested tier by their canonical remaining usage capacity percentage (remaining_pct) and routes the prompt to whichever provider holds the highest available quota headroom.

- **Custom Mode (mode**: "custom"):

Bypasses remaining quota percentages and adheres strictly to a user-defined provider preference list (custom_order, e.g., ["claude", "codex", "agy", "grok"]).

Prompts hit the first available provider in the list; the router only advances down the sequence if the primary provider is offline, stopped, depleted (remaining_pct $<=$ 0), or encounters a pre-stream execution error.

## 4. Client Integration & Per-Task Assignment

Semantic tiers enable a universal integration method across downstream bring-your-own-endpoint (BYOM) software:

Universal Configuration Rule for Routable Apps

Any application supporting custom OpenAI base URLs or Ollama hosts can connect in three steps:

- **Endpoint**: Point to http://127.0.0.1:8090 (Ollama host) or http://127.0.0.1:8090/v1 (OpenAI format).
- **Model Name**: Enter fast, medium, or reasoning.
- **API Key**: Use any placeholder string (e.g., ollama).

Per-Task Model Routing in Client Applications

- **Obsidian (LLM Wiki)**: The karpathywiki plugin uses per-task smart levels

(usePerTaskModels: true) to assign different semantic tiers to distinct background operations within the vault (ingestModel: "fast", lintModel: "reasoning", queryModel: "reasoning").
- **Claudia (Claude Code Second Brain)**: Remaps ANTHROPIC_BASE_URL in .claude/settings.local.json to port 8090, translating Claude Code's requested models directly to Token Buffet's tiers (opus → reasoning, sonnet → medium, haiku → fast).
- **Everyday Consumer Software**: Applications like Brave Leo, Raycast, Home Assistant, Joplin, Logseq, Thunderbird (Ollama Compose), AnythingLLM, and Open WebUI map their chat or completion features directly to these three tiers.

## 5. Operator Control via the Dashboard

The single-page operator console served on port 8080 provides full administrative visibility over semantic tiers:

- **Routing Mode Toggle**: Operators can switch between Usage Mode ("Spread work across accounts") and Custom Mode ("Always use my best account first").
- **Tier Customization & Pinning**: Under Advanced Mode, operators can inspect and edit the tier-to-model mappings directly in router_config.json or re-populate them from provider regexes.
- **Restart Requirement Banner**: Because active router instances retain a startup snapshot of tier mappings to prevent mid-session configuration drift, the dashboard displays a banner reminding operators that routing edits require a router process restart to take effect.

## Usage Mode: Sorts by remaining quota

In the larger context of Smart Routing & Tiers within the Token Buffet AI Subscription Hub, Usage Mode ("mode": "usage") serves as the primary, quota-aware load-balancing engine for the Local Router Gateway (port 8090).

Rather than sending all prompt traffic to a single fixed AI provider, Usage Mode dynamically inspects the remaining usage capacity across all connected subscription accounts and routes incoming requests to whichever backend currently holds the highest available quota headroom.

Below is an in-depth breakdown of how Usage Mode operates, its interaction with semantic tiers, its underlying data sources, and its representation across the operator interface.

## 1. Core Mechanics & Value Proposition

- **Capacity Maximization**: In Usage Mode, the router ranks all active, running backend providers supporting a requested tier by their canonical remaining quota percentage (remaining_pct) and selects the provider with the most headroom.
- **Spreading the Workload**: The primary value proposition for users is the ability to "spread work across accounts". By distributing requests across multiple subscription CLIs (such as Claude Code, Codex CLI, Grok Build, and Gemini Antigravity), users avoid unexpected rate limits on any single account without paying per-token API fees or buying separate developer keys.
- **Contrast with Custom Mode**: Usage Mode contrasts directly with Custom Mode ("mode": "custom"), which ignores remaining quota levels and strictly adheres to a hardcoded preference order (custom_order, e.g., prioritizing Claude before falling back to Codex or Grok).

## 2. Integration with Semantic Tiers & Request Resolution

Usage Mode operates as the second step in Token Buffet's two-stage routing pipeline:

- **Semantic Tier Resolution**: Client applications submit requests using abstract performance tiers—fast (Quick), medium (Balanced), or reasoning (Best)—rather than hardcoded model IDs. The router translates the requested tier into a list of candidate vendor models defined in config/router_config.json (for example, mapping reasoning across Gemini Antigravity, Claude Opus, Codex GPT-6, and Grok-4.7).
- **Quota Sorting & Selection**: Once the eligible providers for that tier are identified, Usage Mode evaluates their live usage exports, sorts the candidates by remaining_pct from highest to lowest, and dispatches the prompt to the top-ranked provider.
- **Automatic Skipping & Failover**: Usage Mode automatically bypasses backends that are stopped, offline, or depleted (remaining_pct $<=$ 0). If the selected backend encounters an error, rate limit (HTTP 429), or CLI connection failure before text generation starts, the router advances seamlessly to the provider with the next-highest remaining quota within that tier.

## 3. Asynchronous Quota Capture & Zero-Latency Execution

To ensure that quota evaluation never adds latency to live API requests, Usage Mode relies on a decoupled data pipeline:

- **Background Extraction**: Individual CLI proxies manage background pseudo-terminal (PTY) or subcommand tasks that run vendor status commands (such as /usage) to capture remaining quota metrics.
- **Decoupled JSON Exports**: Extracted metrics are written asynchronously to disk under data/providers_status/_usage.json. The canonical sorting field is usage.limits..remaining_pct.
- **Cached Polling**: The Smart Router on port 8090 never executes screen-scraping scripts inline during a request. Instead, it polls each active proxy's /usage endpoint on a background schedule (defaulting to every 30 minutes), keeping prompt routing fast and responsive.

## 4. Operator Control & Dashboard Visualization

Usage Mode is fully integrated into the Control Center Dashboard (port 8080):

- **Visual Quota Gauges**: On the Dashboard Home surface and Accounts cards, quota availability is displayed as visual gauges sorted by tightest remaining capacity, reflecting the same quota hierarchy used by Usage Mode.
- **Routing Mode Toggle**: Under the Routing panel in Advanced Mode, operators can toggle between "Spread work across accounts (save quota)" (usage) and "Always use my best account first" (custom).
- **Restart Requirement**: The dashboard explicitly notes that toggling routing modes rewrites config/router_config.json, which requires a router process restart to take effect because active router instances retain a startup snapshot of configuration state.

## Custom Mode: Follows custom order

In the larger context of Smart Routing & Tiers within the Token Buffet AI Subscription Hub, Custom Mode ("mode": "custom") serves as the preference-driven load-balancing strategy for the Local Router Gateway operating on port 8090

While Usage Mode automatically spreads prompt traffic to whichever backend currently holds the highest remaining quota percentage, Custom Mode allows users to bypass quota-equalization logic and direct prompts strictly according to a hardcoded provider preference sequence known as custom_order.

## 1. Core Mechanics & Strategic Purpose

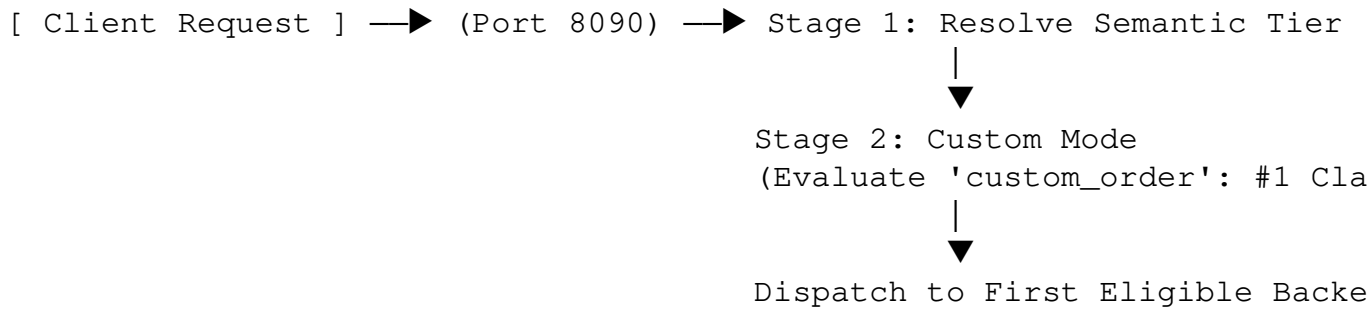
- **Fixed Preference Sequence**: Under Custom Mode, the router evaluates candidate providers based on an ordered array in config/router_config.json (such as custom_order: ["claude", "codex", "agy", "grok"]).

"Use Best First" Philosophy: In user-facing interfaces like the Control Center Dashboard (port 8080), Custom Mode is represented in plain language as "Always use my best account first" or "Use best first".

- **Target Persona & Intent**: Custom Mode is designed for users or operators who prioritize specific model capabilities or vendor preferences (e.g., always preferring Claude Opus or GPT-6 whenever available) and only want to utilize secondary providers when the primary choice is depleted, offline, or rate-limited.

## 2. Integration with the Two-Stage Smart Routing Pipeline

Custom Mode functions as the second phase of Token Buffet's prompt dispatch workflow:




- **Stage 1 — Semantic Tier Resolution**: Client applications submit requests specifying abstract tiers (fast / Quick, medium / Balanced, reasoning / Best) or vendor model IDs. The router maps the requested tier to candidate models across all registered backends defined in router_config.json.
- **Stage 2 — Custom Order Evaluation**: Instead of checking and sorting providers by headroom (remaining_pct), Custom Mode iterates through custom_order from top to bottom. The router checks whether the first-preference provider supports the requested tier, is actively running (run_status = running on its assigned port), and has available quota (remaining_pct > 0).
- **Execution**: If the top-ranked provider is healthy, the prompt is immediately dispatched to its proxy process.

## 3. Fault Tolerance, Skipping, and Pre-Stream Failover

Custom Mode enforces multi-tiered error handling to ensure client requests do not drop if a preferred provider is unavailable:

- **Automatic Depletion Skipping**: Providers marked as stopped, offline, or depleted (remaining_pct <= 0) are automatically skipped in the sequence.
- **Pre-Stream Failover**: If the primary provider in custom_order encounters a connection

failure, an HTTP 429 rate limit, or a CLI execution error before text generation starts, the router advances down the custom_order list to the next candidate provider in the tier.
- **Disk Buffering for Retries**: Non-streaming API requests are buffered in memory and spooled to disk if payload sizes exceed 1 MiB. This guarantees that if a preferred backend fails during execution, the router retries the full prompt cleanly on the next provider in custom_order without partial payload corruption.
- **Exhaustion Fallback**: If every provider listed in custom_order for a given tier is offline, depleted, or returns an error, the router returns an HTTP 503 service unavailable response.

## 4. Operator Experience & Dashboard Controls (:8080)

Operators manage Custom Mode through the Control Center Dashboard running on port 8080:

- **Routing Panel (Advanced Mode)**: Located under the Router tab (revealed via the header "Advanced mode" toggle), operators can switch the system mode between "Spread work across accounts (save quota)" (usage) and "Always use my best account first" (custom).

custom_order Drag & Reorder List: The dashboard provides an interactive list allowing operators to reorder provider priority (e.g., moving claude to #1 or grok to #2).

- **Restart Requirement Banner**: Whenever routing mode or custom_order edits are saved, config/router_config.json is updated on disk. The dashboard displays an explicit restart-required banner because running router instances hold a startup snapshot of configuration state to prevent mid-session drift.
- **Comparison Summary**: Custom Mode vs. Usage Mode

| Dimension | Custom Mode ("custom") | Usage Mode ("usage") |
|---|---|---|
| Primary Selection Rule | Follows strict custom_order sequence | Sorts backends by highest remaining quota % (remaining_pct) |
| User Label in UI | "Always use my best account first" / "Use best first" | "Spread work across accounts (save quota)" |
| Best Used For | Quality-focused workflows where one AI provider is strongly preferred | Quota preservation across multiple accounts to prevent rate limits |
| Fallback Trigger | Moves to next in list only if higher-ranked backend is down, depleted, or errors | Moves to next highest-quota backend if top-headroom backend is unavailable |

## Tool Handoff: Structured tool-call decisions

In the architecture of the Token Buffet AI Subscription Hub, Tool Handoff represents a crucial protocol bridge that allows downstream software applications to leverage complex function calling, agentic tools, and local Model Context Protocol (MCP) servers while running on flat-rate command-line interface (CLI) subscriptions.

When a client application (such as a developer assistant, Obsidian vault plugin, or local second-brain daemon) submits a prompt containing tool or function definitions, the Smart Router Gateway (port 8090) intercepts the request. Rather than allowing the backend vendor CLI to execute tools in its own remote environment or isolated CLI process, the router

requests one structured tool-call decision from the backend model and delivers that decision back to the local client application for execution.

Below is an in-depth breakdown of tool handoff mechanics, provider wire formats, and its role within the larger framework of Smart Routing & Tiers.

## 1. Core Mechanics & Execution Flags (--tools "")

The fundamental objective of Tool Handoff is to separate reasoning and decision-making (handled by the subscription-backed AI model) from action execution (handled safely by the local client application).

- **Structured Tool Decisions**: When an incoming API payload carries tool or function schemas, the gateway passes those definitions to the selected backend vendor CLI. The vendor CLI generates a structured tool-call payload (specifying which function to invoke and supplying the necessary parameters) without running the tool itself.
- **Explicit Execution Flags (tool_handoff_args)**: To prevent the vendor CLI from attempting to execute functions inside its own process, provider configurations (registered_providers//config.json) specify execution.tool_handoff_args. For example, both Claude Code and Grok Build configurations append --tools "" (an empty string argument) when invoking the CLI, signaling to the vendor engine that it should return a tool-choice decision while leaving tool execution strictly to the client.

## 2. Protocol & Wire-Format Capabilities Across Backends

Because different AI vendors use distinct communication protocols, Token Buffet's router translates and buffers tool decisions across four major wire formats:

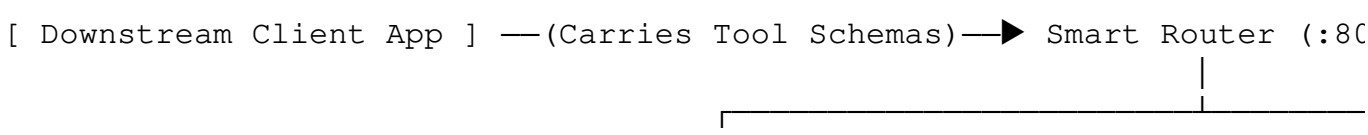
- **Codex CLI (/v1/responses)**: Operating over the OpenAI Responses API, Codex CLI supports buffered function, custom freeform, and apply_patch tool handoff. The gateway holds continuation history for recent responses so multi-step tool calls can continue seamlessly.
- **Grok Build (/v1/chat/completions)**: Utilizing the standard OpenAI Chat Completions wire format, Grok Build supports buffered function-call handoff. During backend processing, it emits periodic empty chunks to keep client connections alive while the model formats its tool decision.

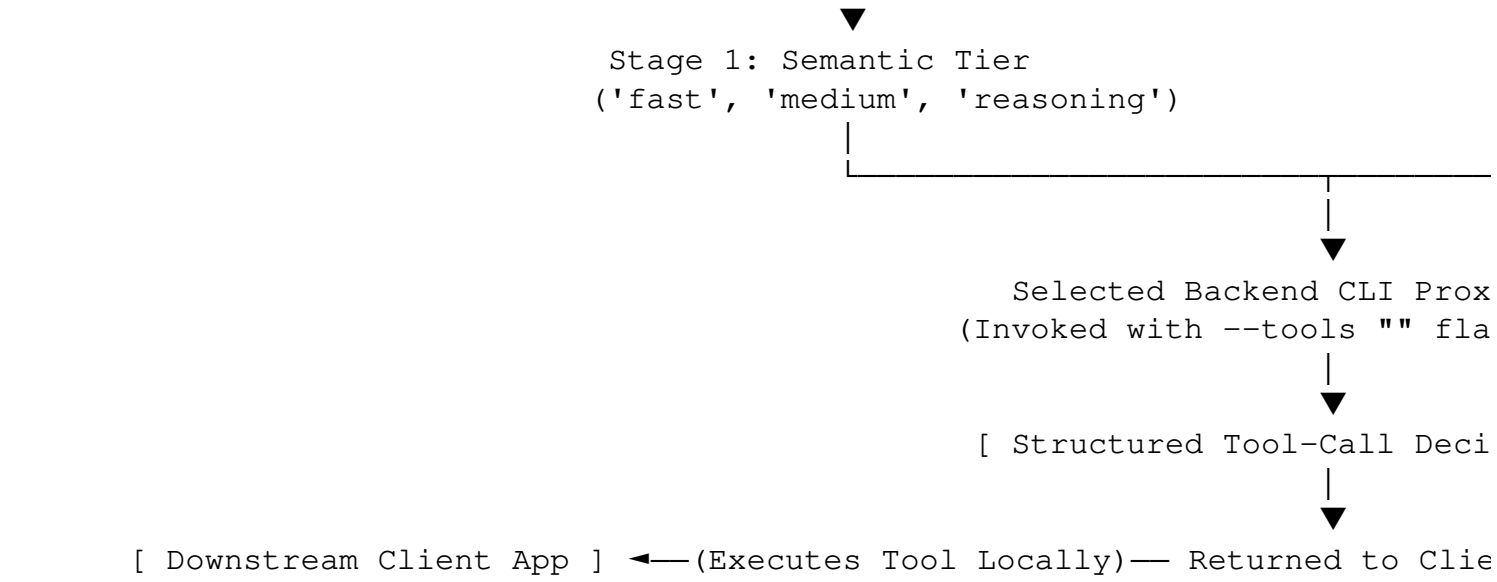
Gemini Antigravity (/v1beta/models/{model}:generateContent): Operating via Gemini's GenerateContent protocol, Antigravity provides buffered function-call handoff for client-declared tools.

- **Claude Code (/v1/messages)**: Communicating over the Anthropic Messages wire format (/v1/messages), Claude Code accepts tool declarations directly and returns structured tool_use content blocks to the client application.

## 3. Tool Handoff in the Larger Context of Smart Routing & Tiers

Tool Handoff operates seamlessly alongside Token Buffet’s Smart Routing and Semantic Tier resolution pipeline:






- **Tier Resolution**: Downstream tools invoke generic performance tiers (fast, medium, or reasoning) rather than fixed vendor model IDs. A complex agentic task requiring multi-tool reasoning can hit the reasoning tier (mapping to models like Claude Opus, GPT-6, or Gemini Pro High).
- **Quota-Aware Dispatch**: The router selects an active backend based on Usage Mode (routing to the provider with the highest remaining quota percentage) or Custom Mode (following a strict preference list).
- **Transparent Protocol Translation**: Regardless of which provider handles the request, the router formats the client's tool definitions into the backend's expected CLI flags, receives the structured tool invocation payload, and packages it back into the wire format requested by the client.

### 4. Client Ecosystem Impact & Practical Workflows

Tool Handoff allows advanced local AI tools to maintain local state and privacy while tapping into subscription compute:

- **Claudia (Claude Code Second Brain)**: Claudia combines a Python memory daemon (~33 MCP tools over SQLite and vector embeddings) with an Obsidian 3D graph visualizer. By pointing ANTHROPIC_BASE_URL to port 8090, Claudia sends tool-heavy requests through the router. The router passes tool calls to the backend subscription model and returns the structured decision back to Claudia to query local SQLite databases or update Obsidian notes locally.
- **Developer Chatbot Workspaces**: Isolated workspaces for Claude Code, Codex CLI, Grok Build, and Gemini Antigravity can request file modifications (apply_patch), shell executions (Bash), or code edits (Edit). The router ensures that tool decisions return to the initiating workspace launcher so actions are executed within the local project directory.

## Integrated Apps

Downstream apps are configured by the same file set at three layers: the base `integrations/`, an app type such as `integrations/obsidian/`, and a registered instance under `registered_apps/<type>/<name>/`. Adding an app is a filesystem change, not an engine change.

## 1. App Types

- **Obsidian**: The karpathywiki plugin (LLM Wiki) points at the router, with per-task tiers such as `ingestModel: "fast"`, `lintModel: "reasoning"`, and `queryModel: "reasoning"`.
- **Claudia**: A Claude Code based second brain that connects by setting `ANTHROPIC_BASE_URL` to the router in `.claude/settings.local.json`.
- **Chatbot**: Claude Code, Codex CLI, Grok Build, and Gemini Antigravity presets under `integrations/chatbot/presets/`.
- **Abject**: A further app type with its own installer.

## 2. Registration

`tbuffet add-app <type> [name] --path <dir>` (or the dashboard wizard, which calls `/api/apps/register`) runs `integrations/register.py`. It copies the type's files into the instance folder, overlays the chosen preset, substitutes placeholders, then runs the instance's `init.py` and `configure.py` to write the app's own settings file.

## 3. Re-applying Settings

`tbuffet reconfigure-app <type> <name>` or the dashboard Re-apply action re-runs `configure.py`, pushing the router endpoint and tiers into the app while leaving its other settings alone.

## 4. Presets

Obsidian vaults choose a preset from `vault_types.json`: `default`, `finance-conference`, `research-papers`, or `daily-notes`. Each preset supplies entity and concept tags and system prompt instructions.

## 5. Opening Apps

An app type's `init.py` can define `check()` (hides uninstalled types), `link()`, and `launch()`. `tbuffet open <type> <name>` and the dashboard Open button use the instance's own `launch()`.

## Obsidian: Karpathywiki notes Q&A

In the ecosystem of the Token Buffet AI Subscription Hub, Obsidian running the karpathywiki plugin serves as the flagship knowledge management integration. Inspired by Andrej Karpathy's "LLM Wiki" paradigm, this setup converts flat-rate command-line interface (CLI) subscriptions—such as Claude Code, Codex CLI, Grok Build, and Gemini Antigravity—into a local knowledge-synthesis engine for personal vaults.

By pointing the plugin's API base URL to Token Buffet's central router gateway on port 8090, users can run automated note ingestion, structured tag extraction, and vault-wide Q&A without paying per-token API fees or purchasing separate developer keys.

Below is an in-depth breakdown of what the sources specify regarding Obsidian's notes Q&A engine, per-task smart routing, template customization, instance architecture, and position within the broader landscape of integrated applications.

## 1. The Karpathywiki Engine & Notes Q&A Mechanics

The karpathywiki plugin transforms an Obsidian vault into an interconnected, synthesis-driven wiki:

- **Automated Document Ingestion**: Raw text, transcripts, or web captures dropped into a vault's inbox folder (_Inbox/) are automatically processed by background tasks. The engine extracts core entities and concepts, links related topic pages using [[wikilinks]], and deduplicates overlapping content.
- **Vault Notes Q&A**: Users can query their vault on any subject. Rather than returning simple keyword search matches, the Q&A engine queries the local router, synthesizes insights across multiple vault documents, cross-links relevant topic pages, and delivers a grounded answer.
- **Target Plugin Configuration**: The integration targets the community plugin settings file located at .obsidian/plugins/karpathywiki/data.json within the user's vault directory.

## 2. Per-Task Model Routing & Semantic Tiers

Obsidian leverages Token Buffet's per-task smart levels ("usePerTaskModels": true) defined inside app_settings.json. Instead of routing all vault actions to a single static AI model, different background tasks trigger distinct semantic model tiers based on computational complexity:

- **Ingestion (ingestModel**: "fast"): High-frequency, low-latency background tasks—such as parsing incoming note text and generating initial entity tags—are assigned to the fast tier (Quick).
- **Vault Linting (lintModel**: "reasoning"): Periodic vault maintenance, tag vocabulary enforcement, and link restructuring are assigned to the reasoning tier (Best).
- **Notes Q&A (queryModel**: "reasoning"): Complex multi-document analysis, comparative synthesis, and vault Q&A queries trigger the reasoning tier (Best).

When a query or ingestion task is submitted, Token Buffet's Smart Router Gateway (port 8090) receives the requested tier and dynamically routes the prompt to whichever connected provider subscription currently holds the highest remaining quota headroom (in Usage Mode) or follows a hardcoded preference list (in Custom Mode).

## 3. Preset Extraction Schemas (vault_types.json)

To tailor extraction and Q&A behavior to specific types of knowledge bases, Obsidian integrations utilize reusable preset templates defined in vault_types.json:

finance-conference: Tailored for financial transcripts and investor materials. Sets custom entity tags (person, organization, fund, asset_class, instrument, index) and custom concept tags (financial_strategy, portfolio_archetype, action_playbook), while supplying specialized system prompt instructions for financial analysis.

research-papers: Optimized for academic literature, citations, methodology synthesis, and paper analysis.

daily-notes: Designed for daily activity logs, personal journal entries, action item tracking, and task summaries.

During registration or reconfiguration, choosing a preset overlays these custom tag schemas and system instructions into the plugin's configuration without requiring manual prompt engineering.

## 4. Configuration Architecture & Instance File Layout

Obsidian vaults are registered and managed through standard CLI utilities or the Control Center Dashboard (port 8080):

- **Registration Command**: Workspaces are registered using tbuffet add-app obsidian --path <VAULT_PATH> --vault-type

or through the 3-step modal wizard on the Dashboard (My Apps tab).

- **Instance Directory**: Each registered vault creates an instance folder under registered_apps/obsidian// containing three configuration files:

app_meta.json: Records the concrete location path of the vault's data.json file and the active vault_type preset.

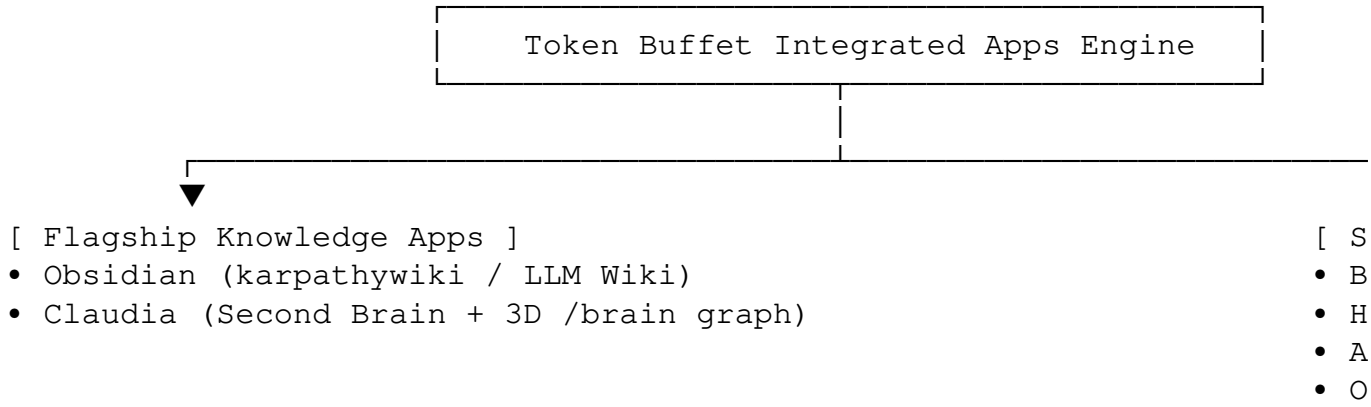
app_settings.json: Stores client endpoints (http://127.0.0.1:8090/v1/), active tiers, per-task smart levels, and token limits.

vault_types.json: Maintains the schema preset definitions.

- **Re-application (reconfigure-app)**: Executing tbuffet reconfigure-app obsidian or clicking Re-apply in the Dashboard re-runs configure.py. This re-pushes updated endpoints and preset tags into .obsidian/plugins/karpathywiki/data.json while preserving all other user plugin settings intact.

## 5. Obsidian in the Larger Context of Integrated Apps

In the broader landscape of Token Buffet's client software integrations, Obsidian plays a pivotal role:




- **Flagship Status**: Obsidian is the fully completed flagship integration (#1 on the consumer routing roadmap) and serves as the baseline model for non-technical application delivery.
- **Complementing Claudia**: While Obsidian runs as a standalone note-taking app with karpathywiki via OpenAI endpoints, it also serves as the local knowledge projection for Claudia. Claudia combines a Claude Code persona, a Python memory daemon (~33 tools), and a 3D brain-graph visualizer (/brain) that projects synthesized topic pages

into an Obsidian vault (~/.claudia/vault/) while routing prompts through port 8090
- **Delivery Tier A Benchmark**: Obsidian exemplifies "Delivery Tier A" (configuring an existing routable app by pointing its endpoint field to port 8090). It proves the core architectural promise: multiple independent knowledge bases running seamlessly on a single, shared subscription backend.

## Claudia: Local second brain persona

In the larger context of the Token Buffet AI Subscription Hub, Claudia represents an advanced, specialized integration that showcases how complex, local "second brain" systems can offload AI computation across flat-rate subscriptions while keeping user memory and knowledge repositories strictly local.

Below is an in-depth analysis of Claudia's architecture, its integration mechanics within Token Buffet, its semantic tier mapping, and its broader role in the integrated app ecosystem.

## 1. What Claudia Is: Persona, Memory Daemon, and 3D Visualizer

Claudia is a local, persona-driven "second brain" assistant built on top of Claude Code:

- **Persona & Skills**: Combines an AI chief-of-staff persona with approximately 45 specialized skills to help users track commitments, manage relationships, analyze patterns, and think strategically.
- **Python Memory Daemon**: Utilizes a Model Context Protocol (MCP) server equipped with roughly 33 tools operating over a local SQLite database and vector embeddings to retain long-term contextual memory.
- **Obsidian Vault Projection**: Projects its synthesized memory into a structured Obsidian knowledge vault located at ~/.claudia/vault/, organized using the PARA layout with interconnected [[wikilinks]] and topic pages.

3D Brain-Graph Visualizer: Incorporates the /brain 3D brain-graph visualizer ("graphify") to interactively render and explore the conceptual connections across the local knowledge graph.

## 2. The Core Problem & Token Buffet's Wiring Solution

- **Upstream Constraint**: Stock upstream Claudia is locked exclusively to Claude Code and possesses no native model routing or multi-provider support.
- **Direct Wire Routing**: Token Buffet adds a routing layer by setting ANTHROPIC_BASE_URL to http://127.0.0.1:8090 inside the workspace's .claude/settings.local.json file.
- **No Intermediate Hop Needed**: Because Token Buffet's Local Router Gateway natively understands and handles the Anthropic Messages wire format on /v1/messages, Claudia connects directly to port 8090 without requiring intermediate translation layers like LiteLLM.

Claudia (Claude Code)⟶Token Buffet Router (:8090)⟶CLI Proxy⟶Vendor CLI

## 3. Semantic Tier Translation

When Claudia issues requests for different Claude models, Token Buffet automatically maps those requests directly to its abstract semantic performance tiers:

opus → reasoning (Best): Reserved for deep multi-document reasoning, complex strategic synthesis, and memory consolidation.

sonnet → medium (Balanced): The default tier for standard conversation, drafting, and skill execution.

haiku → fast (Quick): Utilized for high-frequency, low-latency background memory lookups and quick tag extraction.

Once mapped to a tier, Token Buffet's Smart Router dispatches the request to whichever connected vendor subscription (Claude, Codex, Gemini Antigravity, or Grok) currently holds the highest remaining usage quota headroom (in Usage Mode) or follows the user's custom preference sequence (in Custom Mode).

## 4. Registration, Configuration & Instance Architecture

Claudia instances are managed through Token Buffet's standardized integrations/claudia/ application framework:

- **Registration Command**: A new Claudia instance is registered via the command line or dashboard using:

tbuffet add-app claudia "My Claudia" --path "/path/to/claudia"

- **Workspace File Layout**: Registration creates an isolated instance directory under registered_apps/claudia/ containing:

app_config.json: Defines the workspace root path and initial client settings.

app_settings.json: Configures ANTHROPIC_BASE_URL as http://127.0.0.1:8090 in .claude/settings.local.json.

init.py: Writes and merges the local gateway redirect into the target workspace's .claude/settings.local.json.

- **Launcher & Control Center**: The launcher hook enables the Open action button in the Control Center Dashboard (http://127.0.0.1:8080). Clicking Open launches a visible terminal window (PowerShell on Windows or Apple Terminal via AppleScript on macOS) with environment overrides pre-applied.
- **Pre-configured Demo**: Token Buffet includes a pre-registered demo instance located at registered_apps/claudia/My-Client-Relationships-Management-Demo/.

## 5. Position in the Integrated Apps Ecosystem & Current Limitations

Within the broader landscape of integrated applications, Claudia occupies a unique position alongside Obsidian:

- **Karpathy-Style Wiki Synergies**: Both Claudia's vault and the Obsidian LLM Wiki plugin adhere to Karpathy-style knowledge synthesis patterns. Together, they showcase the "second brain + wiki" story: Claudia provides an active assistant with a 3D brain graph, while Token Buffet powers the underlying AI engine end-to-end.
- **Local Privacy + Offloaded Compute**: Local SQLite databases, vector embeddings, and Obsidian vault notes remain entirely private on the user's machine, while the heavy LLM text generation is offloaded across subscription CLIs.

- **Current Open Items & Known Limits**:
- **MCP Tool Compatibility**: Whether non-Claude backends like Gemini Antigravity (agy) fully support Claudia's MCP tool stream protocol remains an unverified open item.
- **Multimodal Ingestion**: Direct PDF and image ingestion inside Claudia sessions is currently blocked until multimodal proxy forwarding is implemented.

## Workspace Presets: Configurable app integrations

In the architecture of the Token Buffet AI Subscription Hub, Workspace Presets and Configurable App Integrations provide the template-driven foundation that connects client software—such as Obsidian knowledge vaults, Claudia second-brain workspaces, and developer chatbot CLIs—to the central API gateway (port 8090).

Rather than hardcoding vendor or application logic into the router or dashboard, Token Buffet adopts a config-driven, filesystem-first ethos. Integration types are defined as template directories under integrations// (or integrations/chatbot/presets/), and registered instances are maintained in registered_apps///. This design allows users to register, configure, and reconfigure multiple independent workspaces while tailoring their extraction behavior, prompt schemas, and model routing to specific tasks.

## 1. The Instance File Structure (registered_apps///)

When an application or workspace is registered, the system creates an isolated instance directory containing three core JSON configuration files that drive its behavior:

app_meta.json / app_config.json:

Records the concrete location path of the client application (e.g., vault folder or workspace root).

Tracks the bridge configuration target file (e.g., .obsidian/plugins/karpathywiki/data.json for Obsidian or .claude/settings.local.json for Claudia).

Stores active preset template metadata (such as the selected vault_type).

app_settings.json:

Maintains active client-side settings pushed into the target application, including the gateway endpoint (http://127.0.0.1:8090/v1/), active semantic model tiers (fast, medium, reasoning), and token limits.

- **Defines per-task smart levels ("usePerTaskModels"**: true), allowing different background actions within the app to trigger different model tiers (e.g., using fast for document ingestion while assigning reasoning to complex Q&A and periodic linting).

vault_types.json:

Stores schema definitions, custom entity/concept tag vocabularies, and specialized system prompt instructions for that instance.

## 2. Registration & Reconfiguration Pipeline (register.py & configure.py)

The lifecycle of a workspace preset is managed through a standardized two-stage execution pipeline:

```
[ User / Dashboard / CLI ]
          |
          ▼
```

- **Stage 1**: Registration (integrations/register.py)

```
├── Copies integrations/<type>/ template files ➔ registered_apps/<ty
├── Overlays selected preset (vault_types or chatbot preset)
├── Substitutes placeholders ([app_path], [vault_type]) in app_meta.
└── Runs init.py (scaffolds folders) & configure.py (pushes settings
          |
          ▼
```

- **Stage 2**: Reconfiguration (tbuffet reconfigure-app / Dashboard 'Re-apply')

```
└── Re-executes instance's configure.py ➔ updates client app config
```

Registration (tbuffet add-app or integrations/register.py):

Invoked via the command line or the Dashboard wizard.

Copies base files from integrations// into registered_apps///.

Overlays files from the selected preset directory (if presets_directory is declared).

Substitutes path placeholders (like [app_path] and [vault_type]) in app_meta.json.

Executes init.py to scaffold required workspace folders (e.g., _Inbox/ or local .claude/ directories) and runs configure.py to write settings directly into the target client app's native configuration file.

- **Reconfiguration (tbuffet reconfigure-app / "Re-apply")**:

Allows operators or users to edit app_settings.json or vault_types.json and re-run configure.py.

Pushes updated endpoints, model tiers, or custom extraction tags directly into the target application's settings file while preserving other non-Token-Buffet user preferences intact.

## 3. Key Categories of Workspace Presets

The sources detail several specialized preset types across different application domains:

### A. Obsidian Knowledge Vault Presets (vault_types.json)

Obsidian integrations utilize preset schemas stored in vault_types.json to customize how incoming notes and transcripts are parsed and synthesized:

default: Baseline entity and concept extraction schema.

finance-conference: Tailored for financial transcripts and investor presentations. Sets custom entity tags (person, organization, fund, asset_class, instrument, index), custom concept tags (financial_strategy, portfolio_archetype, action_playbook), and provides specialized system prompt instructions for financial analysis.

research-papers: Optimized for academic literature, citations, methodology comparison, and paper synthesis.

daily-notes: Designed for personal journals, daily logs, task tracking, and periodic summaries.

**B. Chatbot CLI Workspace Presets (integrations/chatbot/presets/)**

Developer chatbot CLIs function as client workspaces using built-in presets (claude, codex, grok, and agy):

Each preset targets specific configuration files—such as .claude/settings.local.json for Claude Code, .codex/config.toml for Codex CLI, .grok/config.toml for Grok Build, and .gemini/antigravity-cli/settings.json for Gemini Antigravity.

Launchers set custom session configuration homes (e.g., CODEX_HOME, GROK_HOME, or GOOGLE_GEMINI_BASE_URL) to isolate project workspaces and prevent native CLI database locks or global configuration overwrites.

**C. Claudia "Second Brain" Workspace (integrations/claudia/)**

Configures Claudia's Claude Code session by setting ANTHROPIC_BASE_URL to http://127.0.0.1:8090 inside .claude/settings.local.json.

Allows Claudia's persona, Python memory daemon (MCP server with ~33 tools over SQLite/vector embeddings), and 3D /brain graph visualizer to route prompts across all connected subscriptions.

### 4. Management via CLI and the Control Center Dashboard

Workspace presets are designed for easy setup by both technical operators and everyday non-technical users:

- **CLI Management**:

# Register an Obsidian vault with the finance-conference preset

tbuffet add-app obsidian my-vault --path "/path/to/vault" --vault-type finance-conference

## Chatbot & CLI Clients

Token Buffet registers vendor CLI agents as router clients, so each one can run in its own workspace while using the pooled subscriptions. Every client points at the router root, `http://127.0.0.1:8090`, like it would at an Ollama server.

### 1. The Four Clients

| Client | Configuration | Router Endpoint | Not |
|---|---|---|---|

| | | | |
|---|---|---|---|
| Claude Code | `.claude/settings.local.json` (passed with `--settings`) | `/v1/messages` (Anthropic Messages) | Sets `CLAUDE_CODE_DISA` and enables gateway |
| Codex CLI | `.codex/config.toml` (`CODEX_HOME` set to the workspace) | `/v1/responses` (OpenAI Responses) | Needs the `openai` SI environment |
| Grok Build | `.grok/config.toml` (`GROK_HOME` set to the workspace) | `/v1/chat/completions` (OpenAI Chat) | Emits periodic empty the connection open |
| Gemini Antigravity (agy) | `.gemini/antigravity-cli/settings.json` (`GOOGLE_GEMINI_BASE_URL`) | `/v1beta/models/{model}:generateContent` (Gemini) | Backend proxy uses a home at `data/prov` |

## 2. Registration

Register a client with `tbuffet add-chatbot <cli> --path <folder>`, or from the dashboard's My Chatbots tab, then open it with `tbuffet open chatbot <name>`. Launchers scope settings and home overrides to the chosen workspace, so global CLI setups are untouched.

## 3. Loop Prevention

When a client workspace sends a prompt to the router, the router excludes that client's own backend proxy from the candidates to avoid a loopback.

## 4. Tool Handoff

Each client receives one structured tool-call decision per request and executes the tool locally. Responses API continuation history (`previous_response_id`) is held in router memory for about 100 recent responses and is lost on restart.

## Claude Code: Anthropic Messages endpoint

Within the Token Buffet AI Subscription Hub, developer command-line interfaces (CLIs) function both as backend provider engines and as downstream client tools. Among the supported developer tools, Claude Code occupies a key position due to its integration using the Anthropic Messages endpoint (/v1/messages).

By leveraging project-specific configuration files and session environment overrides, Token Buffet enables Claude Code sessions to route prompts directly through its local gateway (http://127.0.0.1:8090) without altering the user's global system configuration or incurring per-token developer API fees.

## 1. Claude Code Wiring & Protocol Details

Token Buffet interfaces with Claude Code by acting as a local Anthropic-compatible API gateway:

- **Bridge Endpoint & Wire Format**: Claude Code communicates over the Anthropic Messages API (/v1/messages), issuing requests formatted as POST /v1/messages?beta=true. During session initialization or health checks, it may also probe HEAD /api/

hello.

- **Configuration Mechanism**: Rather than editing global settings (~/.claude), Token Buffet creates a project-local configuration at .claude/settings.local.json. Workspace launchers explicitly pass this configuration using the --settings parameter.
- **Environment Flags & Model Discovery**:
- **Disabled Internal Thinking**: The workspace launcher injects CLAUDE_CODE_DISABLE_THINKING = 1 to disable native thinking modes.
- **Gateway Model Discovery**: It enables gateway model discovery so executing the /model command inside the Claude Code CLI cleanly populates and lists Token Buffet's available semantic model tiers.
- **Tool Handoff Mechanics**: When Claude Code issues prompts containing client-side tool declarations, Token Buffet configures the backend using execution.tool_handoff_args set to --tools "". This signals the vendor engine to return a structured tool-call decision while leaving actual tool execution safely to the client workspace.

## 2. Comparison Across the Four Core Chatbot & CLI Clients

Token Buffet provisions isolated workspace launchers for four distinct developer chatbot CLIs:

| Client | Configuration File & Workspace Mechanism | Bridge API Endpoint | Protocol & Execution C |
|---|---|---|---|
| Claude Code | .claude/settings.local.json (passed explicitly with --settings). | /v1/messages (Anthropic Messages). | Disables internal thinking (CLAUDE_CODE_DISABLE_ enables gateway model di model, and uses --tools "" f |
| Codex CLI | .codex/config.toml (session CODEX_HOME set to workspace). | /v1/responses (OpenAI Responses). | Requires the openai Pytho proxy environment; suppo function, custom freeform apply_patch tool handoff h up to 100 responses. |
| Grok Build | .grok/config.toml (session GROK_HOME set to workspace). | /v1/chat/completions (OpenAI Chat). | Emits periodic empty chur backend processing to kee sockets active during long |
| Gemini Antigravity (agy) | .gemini/antigravity-cli/settings.json (session GOOGLE_GEMINI_BASE_URL). | /v1beta/models/{model}:generateContent (Gemini API). | Uses an independent back home at data/provider_ho backend proxies and client isolated. |

To prevent infinite loopback calls when a chatbot client workspace issues a prompt, the Smart Router enforces an originating CLI exclusion rule, automatically bypassing that chatbot's own backend proxy during model dispatch.

## 3. Specialized Integration: Claudia ("Second Brain")

The Anthropic Messages endpoint on port 8090 plays an essential role in Token Buffet's integration with Claudia, an advanced local "second brain" assistant:

- **What Claudia Is**: Claudia is a local assistant built on top of Claude Code, combining a persona, ~45 skills, a Python memory daemon (MCP server with ~33 tools over SQLite

and vector embeddings), and an Obsidian vault projection visualized through a /brain 3D brain-graph.
- **Direct Wire Translation**: While stock upstream Claudia is locked exclusively to Claude Code without model management, Token Buffet remaps ANTHROPIC_BASE_URL inside Claudia's .claude/settings.local.json to http://127.0.0.1:8090.
- **Zero Intermediate Translation Hops**: Because Token Buffet's router natively handles the Anthropic Messages wire format on /v1/messages, Claudia connects directly to the router without needing intermediate translation tools like LiteLLM.
- **Semantic Tier Mapping**: Claudia's requested Claude models are translated directly into Token Buffet's semantic performance tiers:
- opus → reasoning (Best)
- sonnet → medium (Balanced)
- haiku → fast (Quick)

## 4. Registration & Workspace Management

Adding and operating Claude Code workspaces is managed seamlessly through command-line utilities or the Control Center Dashboard:

- **CLI Registration**: Workspaces are registered using commands such as tbuffet add-app claudia "My Claudia" --path "/path/to/claudia" or tbuffet add-chatbot claude --path /path/to/workspace.
- **Dashboard Management ("My Chatbots")**: From the Dashboard at http://127.0.0.1:8080, the My Chatbots tab provides dedicated connection controls (such as the "Connect Claude-Code instance" button). The interface prompts the user for a workspace folder path and a display label, validating that no folder is registered twice for the same CLI.
- **Terminal Launchers**: Clicking Open launches a visible terminal window (using PowerShell on Windows or AppleScript via Apple Terminal on macOS) with the appropriate workspace session overrides applied.

## Codex CLI: OpenAI Responses API

Within the Token Buffet AI Subscription Hub, Codex CLI functions as both a backend subscription provider engine and a client workspace. When operating as a client, Codex routes prompts through Token Buffet's central router (http://127.0.0.1:8090), allowing developers to utilize paid OpenAI/ChatGPT subscriptions without paying per-token API fees or altering their global CLI setup.

Below is a detailed breakdown of how Codex CLI integrates with Token Buffet through the OpenAI Responses API and where it fits within the broader ecosystem of chatbot and CLI clients.

## 1. Wire Format & The OpenAI Responses API (/v1/responses)

Unlike traditional Chat Completions endpoints, Codex CLI interacts with Token Buffet using OpenAI's Responses API endpoint:

- **Endpoint**: Communicates over POST /v1/responses.
- **SDK Dependency**: Requires the openai Python SDK to be installed in the proxy environment. If the SDK is missing, the backend proxy cleanly returns an HTTP 503 error to the client.

- **Continuation History**: To support long, multi-step conversational interactions, the local router holds continuation history for up to 100 recent responses in memory. (Note: Because this state is stored in memory, continuation history is reset if the router process restarts

).

## 2. Workspace Configuration & Session Isolation (CODEX_HOME)

Standard vendor CLIs often enforce global user configurations (such as ~/.codex), which can lead to configuration conflicts or database lockups during concurrent runs. Token Buffet overcomes this through session-level environment overrides:

- **Configuration Target**: Settings are stored locally within the project workspace inside .codex/config.toml.
- **CODEX_HOME Override**: When launching a Codex CLI session, the workspace launcher dynamically sets the environment variable CODEX_HOME to point directly to that workspace's .codex folder.
- **Non-Invasive Execution**: Because the router endpoint is loaded strictly within the workspace's session environment, global user settings remain entirely untouched.
- **Terminal Launchers**: Clicking Open on a registered Codex instance in the Dashboard launches a visible terminal window (using PowerShell on Windows or AppleScript via Apple Terminal on macOS) with workspace environment overrides applied.

## 3. Tool Handoff Capabilities

Codex CLI supports advanced function calling and file editing through Token Buffet's structured tool handoff protocol:

- **Buffered Function Handoff**: Receives structured function-call decisions formatted by the model and delivers them back to the client workspace for execution.
- **Custom Freeform Tools**: Handles non-standard client tool declarations passed through the Responses API.

apply_patch Support: Supports native file modification commands (such as applying code diffs across project files) directly within the developer workspace.

## 4. Larger Context Across Chatbot & CLI Clients

Token Buffet provisions isolated workspaces for four primary developer chatbot CLIs, each tailored to a specific vendor protocol:

| Client | Configuration Mechanism | Bridge Endpoint | Key Protocol Fe |
|---|---|---|---|
| Claude Code | .claude/settings.local.json (via --settings) | /v1/messages (Anthropic) | Disables internal thinking (CLAUDE_CODE_DISABLE and enables gateway mod /model. |
| Codex CLI | .codex/config.toml (CODEX_HOME override) | /v1/responses (OpenAI Responses) | Requires openai SDK; supp function, custom freeform apply_patch handoffs with history. |

| | | | |
|---|---|---|---|
| Grok Build | .grok/config.toml (GROK_HOME override) | /v1/chat/completions (OpenAI Chat) | Emits periodic empty chu backend work to keep clie sockets active. |
| Gemini Antigravity | .gemini/antigravity-cli/ settings.json (GOOGLE_GEMINI_BASE_URL) | /v1beta/models/ {model}:generateContent (Gemini API) | Operates an independent s data/provider_homes/agy backend proxies from clie |

Key Ecosystem Rules for Chatbot Clients

- **Originating CLI Exclusion**: When a Codex CLI workspace sends a prompt to port 8090, the Smart Router automatically excludes Codex's own backend proxy from candidate providers to prevent infinite loopback requests.
- **Workspace Management**: Workspaces are registered via the CLI (tbuffet add-chatbot codex --path ) or through the My Chatbots tab in the Control Center Dashboard (http://127.0.0.1:8080).

## Gemini Antigravity: GenerateContent endpoint

Within the Token Buffet AI Subscription Hub, Gemini Antigravity (referred to internally as the agy preset or provider) is one of the four primary command-line interface (CLI) tools integrated as both a backend subscription provider and a client application.

By wrapping Google's native GenerateContent API endpoint (/v1beta/models/{model}:generateContent) and configuring session-level environment overrides, Token Buffet allows developers and local applications to route requests through Google's Gemini models via the central gateway (http://127.0.0.1:8090) without incurring per-token API fees or altering system-wide CLI installations.

Below is an in-depth analysis of what the sources cover regarding Gemini Antigravity's protocol implementation, workspace isolation mechanics, tool handoff capabilities, and role within the broader chatbot client ecosystem.

## 1. Wire Format & Protocol Details (/v1beta/models/{model}:generateContent)

Unlike OpenAI-compatible endpoints (/v1/chat/completions) or Anthropic Messages endpoints (/v1/messages), Gemini Antigravity communicates using Google's native Gemini API structure:

- **Bridge API Endpoint**: Requests are sent over the Gemini GenerateContent wire format at POST /v1beta/models/{model}:generateContent.
- **Environment-Based Custom Endpoint**: Antigravity natively supports specifying a custom API base URL via the GOOGLE_GEMINI_BASE_URL environment variable.
- **Session-Scoped Redirection**: When a Gemini Antigravity chatbot workspace is launched, its workspace launcher sets GOOGLE_GEMINI_BASE_URL to point strictly to Token Buffet's router on port 8090 (http://127.0.0.1:8090) for that specific session.

## 2. Workspace Mechanics & Session Isolation

Standard vendor CLIs often enforce global user configuration directories (such as ~/.gemini), which can cause database lockups or profile corruption during concurrent execution. Token Buffet circumvents these conflicts through multi-layered session environment isolation:

- **Workspace Settings File**: Client configuration for Antigravity workspaces is maintained locally in .gemini/antigravity-cli/settings.json inside the project folder.
- **Session Home Directory**: The workspace launcher sets the session home path directly to the workspace folder, ensuring that settings apply exclusively to that project session without altering global CLI defaults.
- **Isolated Backend Sign-In Home (data/provider_homes/agy)**: To prevent credential overlap or state pollution between client demo workspaces and backend proxy instances, the agy backend proxy uses a dedicated, isolated home directory at data/provider_homes/agy. This directory requires its own independent login authentication from the client workspace.

## 3. Tool Handoff & Function Calling

When a client application or developer workspace issues a prompt containing function declarations or client-side tools, Token Buffet processes the execution through structured tool handoff:

- **Buffered Function-Call Handoff**: Antigravity supports buffered function-call responses formatted according to Gemini's schema requirements.
- **Decision Delegation**: The Smart Router receives tool definitions from the client, requests one structured tool-choice decision from the backend Gemini model, and returns that decision payload to the client application.
- **Local Action Execution**: The actual execution of the tool (e.g., running shell commands, modifying local files, or querying local databases) is handled safely by the downstream client workspace rather than inside the proxy process.
- **MCP Protocol Verification**: While Antigravity supports structured tool handoffs, the documentation notes that whether the agy backend fully supports Claude-style Model Context Protocol (MCP) tool streams (such as those used by the Claudia second-brain assistant) remains an unverified open item.

## 4. Comparison Across the Four Core Chatbot & CLI Clients

Token Buffet provisions isolated workspace launchers for four distinct developer chatbot CLIs, each tailored to its native API wire format:

| Chatbot Client | Configuration Mechanism | Bridge API Endpoint | Key Protocol & Exe Characteristi |
|---|---|---|---|
| Gemini Antigravity (agy) | .gemini/antigravity-cli/settings.json (session home set to workspace; GOOGLE_GEMINI_BASE_URL) | /v1beta/models/{model}:generateContent | Uses independent backend provider_homes/agy); supp function-call handoff. |
| Claude Code | .claude/settings.local.json (explicitly passed via --settings) | /v1/messages (Anthropic Messages) | Disables internal thinking (CLAUDE_CODE_DISABLE_ and enables gateway mode /model. |
| Codex CLI | .codex/config.toml (CODEX_HOME set to workspace) | /v1/responses (OpenAI Responses) | Requires openai Python SD function, freeform, and app handoffs with 100-respons |
| Grok Build | .grok/config.toml (GROK_HOME set to workspace) | /v1/chat/completions (OpenAI Chat) | Emits periodic empty chun backend processing to prev connection socket timeouts |

## 5. Originating CLI Exclusion Rule

To prevent infinite loopback loops across the local network, the Smart Router enforces an originating CLI exclusion rule:

When an Antigravity chatbot workspace submits a prompt to port 8090, the router automatically excludes agy's own backend proxy from the list of candidate providers.

The prompt is routed to an alternate eligible provider (such as Claude, Codex, or Grok) based on the active routing mode (Usage Mode or Custom Mode) and semantic performance tier.

## 6. Semantic Tier Resolution & Operator Dashboard Management

Gemini Antigravity is fully integrated into Token Buffet's tier resolution and fleet monitoring subsystems:

- **Semantic Model Tier Resolution**: In config/router_config.json, the agy provider is mapped to specific model IDs across semantic performance tiers. For example, under the reasoning tier (Best), the router maps agy to gemini-3.1-pro-high.
- **Workspace Registration**: Users register Antigravity chatbot workspaces via the command line using tbuffet add-chatbot agy --path or through the My Chatbots tab on the Control Center Dashboard (http://127.0.0.1:8080).
- **Quota & Service Status**: On the Dashboard Home and Accounts panels, Gemini Antigravity displays live status indicators (🟢 connected) and period-based quota gauges (tracking gemini_5h and gemini_weekly usage limits and reset countdowns).
- **Terminal Launchers**: Clicking Open next to an Antigravity instance in the Dashboard launches a visible terminal window (using PowerShell on Windows or AppleScript via Apple Terminal on macOS) with GOOGLE_GEMINI_BASE_URL and session home environment overrides pre-applied.

## Grok Build: OpenAI chat completions

Within the Token Buffet AI Subscription Hub, Grok Build functions as both a backend subscription provider engine and a client workspace for developers. When configured as a chatbot client, Grok Build routes its prompts through Token Buffet's central API gateway on port 8090 (http://127.0.0.1:8090), allowing developers to harness xAI's Grok models without paying per-token developer API fees or altering system-wide CLI settings.

Below is a detailed breakdown of what the sources specify regarding Grok Build's reliance on the OpenAI Chat Completions wire format, its workspace configuration mechanics, its tool handoff protocol, and its position within the broader ecosystem of chatbot and CLI clients.

## 1. Wire Format & Protocol Details (/v1/chat/completions)

Grok Build communicates with the local router using the standard OpenAI Chat Completions API format:

- **Bridge Endpoint**: Requests are sent over the POST /v1/chat/completions wire format.
- **Keep-Alive Chunking**: To prevent HTTP connection sockets from timing out during long text generations or heavy reasoning tasks, Grok Build's proxy handler emits periodic empty chunks while the backend CLI processes the request. This keeps the socket active until the model delivers its completed output or streaming delta.

- **Buffered Function-Call Handoff**: Supports structured tool-call responses formatted according to standard OpenAI function-calling specifications.

## 2. Workspace Configuration & Session Isolation (GROK_HOME)

Standard vendor CLIs often enforce global user configuration paths (such as ~/.grok), which can cause database locks or settings conflicts when multiple sessions run concurrently. Token Buffet resolves this through session environment overrides:

- **Configuration File**: Workspace settings are stored in .grok/config.toml within the client project folder.
- **Environment Override**: When launching a Grok Build workspace, the launcher sets the session environment variable GROK_HOME to point directly to that workspace's .grok directory.
- **Non-Invasive Execution**: Because settings are scoped exclusively to that session's environment, the user's global system CLI installation remains completely untouched.
- **Terminal Launching**: Launching a Grok workspace from the Control Center Dashboard opens a visible terminal window (using PowerShell on Windows or AppleScript via Apple Terminal on macOS) with GROK_HOME and router endpoint overrides applied.

## 3. Tool Handoff Mechanics (--tools "")

When a developer in a Grok Build workspace submits a prompt containing client tool schemas or function declarations, Token Buffet manages execution via structured tool handoff:

- **Decision Delegation**: The Smart Router forwards the tool definitions to the backend model to obtain a structured tool-call decision.
- **Execution Flags**: Provider definitions configure execution.tool_handoff_args with --tools "" (an empty string argument) when invoking the CLI. This signals the Grok backend engine to return a structured tool decision without attempting to run the tool inside the backend proxy process.
- **Client Execution**: The router receives the structured tool payload from Grok and delivers it back to the client workspace, ensuring local file modifications, terminal commands, or code edits are executed safely by the local client application.

## 4. Comparison Across Chatbot & CLI Clients

Token Buffet provisions isolated workspaces for four primary developer chatbot CLIs, each tailored to its native API wire format:

| Chatbot Client | Configuration Mechanism | Bridge API Endpoint | Key Protocol & Execut |
|---|---|---|---|
| Grok Build | .grok/config.toml (GROK_HOME override) | /v1/chat/completions (OpenAI Chat) | Emits periodic empty chu generations; uses --tools "" function handoffs. |
| Claude Code | .claude/settings.local.json (passed via --settings) | /v1/messages (Anthropic Messages) | Disables internal thinking (CLAUDE_CODE_DISABLE and enables gateway mod /model. |

| | | | |
|---|---|---|---|
| Codex CLI | .codex/config.toml (CODEX_HOME override) | /v1/responses (OpenAI Responses) | Requires openai Python Sl function, freeform, and ap handoffs holding 100-resp |
| Gemini Antigravity (agy) | .gemini/antigravity-cli/ settings.json (GOOGLE_GEMINI_BASE_URL) | /v1beta/models/ {model}:generateContent (Gemini API) | Operates an independent l (data/provider_homes/agy backend proxies from clie |

### 5. Originating CLI Exclusion Rule & Registration

To ensure smooth operation across the local network, Token Buffet enforces two essential operational rules for Grok Build:

- **Originating CLI Exclusion**: When a Grok Build chatbot workspace sends a prompt to port 8090, the Smart Router automatically excludes Grok's own backend proxy from the list of candidate providers to prevent infinite loopback requests.
- **Workspace Registration**: Workspaces can be registered via the command line using tbuffet add-chatbot grok --path

or through the My Chatbots tab on the Control Center Dashboard (http://127.0.0.1:8080).

## Potential Consumer Integrations

In the architecture of the **Token Buffet AI Subscription Hub**, **Potential Consumer Integrations** represent the strategic expansion layer designed to bring flat-rate, subscription-powered AI to regular, non-technical everyday users. While developer tools focus on terminal automation and coding, consumer integrations demonstrate how interactive command-line interface (CLI) subscriptions—such as **Claude Code**, **Codex CLI**, **Grok Build**, and **Gemini Antigravity (`agy`)**—can power everyday desktop applications, browser assistants, voice agents, and personal knowledge bases without requiring per-token developer API keys or incurring usage bills.

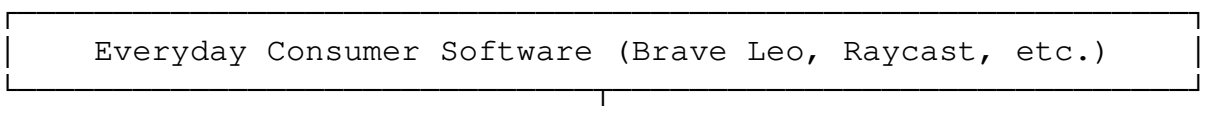
An analysis of the local AI application landscape reveals how Token Buffet bridges subscription quotas into consumer software, the hard boundaries of routable versus cloud-locked tools, the top consumer routing targets, and the three-tier delivery framework designed for everyday users.

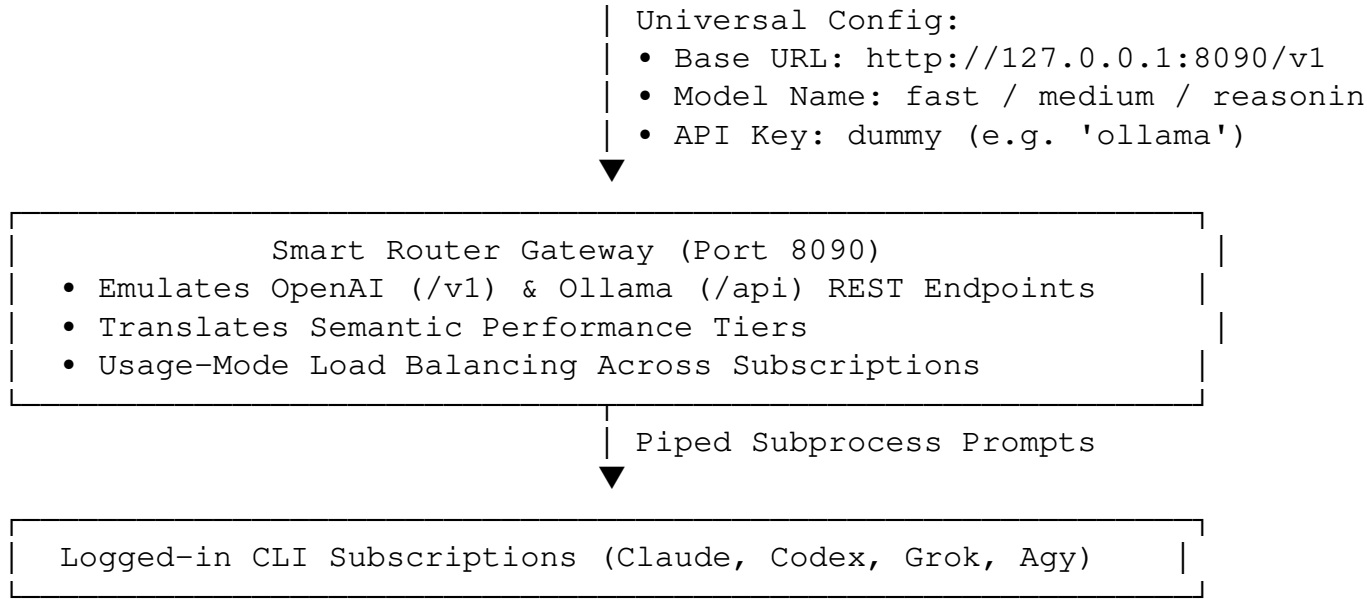
---

### 1. The Strategic Imperative & Universal Configuration Rule

The local AI landscape has structurally shifted from generic, standalone conversational windows toward domain-embedded inference engines where LLMs operate behind native UI triggers, global hotkeys, and automated event pipelines. However, regular consumers who pay monthly fees for cloud AI subscriptions (such as Claude, ChatGPT, Gemini, or Grok) are typically unable to use those subscriptions inside third-party desktop tools because vendor web platforms do not expose developer API keys.

Token Buffet solves this through its **Universal Configuration Rule**:






By serving both **Ollama** (`/api/chat`, `/api/generate`) and **OpenAI-compatible** (`/v1/chat/completions`) endpoints on **port 8090**, any application that supports custom base URLs or Ollama server configurations can connect with a two-line setup. Downstream software requests generic **semantic performance tiers—`fast`** (Quick), **`medium`** (Balanced), or **`reasoning`** (Best)—and Token Buffet's Smart Router dynamically dispatches each query to whichever logged-in subscription currently holds the highest available quota headroom.

---

## 2. The Honest Boundary: Cloud-Locked vs. Routable Applications

To set user expectations accurately, Token Buffet establishes a strict distinction between cloud-locked platforms and routable consumer software:

- **Cloud-Locked Consumer Apps (Cannot Route)**: Major mainstream web platforms—including ChatGPT web, Gemini web, Claude.ai web, Perplexity Pro, Canva Magic, CapCut, Grammarly, Notion AI, Otter.ai, and the Opera browser (which includes local LLMs but provides no custom API endpoint setting)—are vendor-locked and cannot be re-pointed to local bridges.
- **The Routable Universe**: Token Buffet focuses exclusively on software built around a **"Bring Your Own Model" (BYOM)** or custom endpoint architecture, allowing users to power their favorite open-source or desktop software with flat-rate subscription quotas.

---

## 3. Top 10 Consumer Routable Targets Roadmap

Token Buffet prioritizes consumer targets based on installation reach, value-to-effort ratios, and native API flexibility:

| # | Application | Category / Built-in Feature | Connection Mechanism | Fit & Strategic Value |
|---|---|---|---|---|
| 1 | **Obsidian** | Knowledge Vault, Notes Q&A & Wikilinks | `karpathywiki` plugin → `http://127.0.0.1:8090` | **★★★ Flagship Target (Completed)** |

| | | | | |
|---|---|---|---|---|
| 2 | **Brave Browser (Leo)** | BYOM Browser Assistant: Page Summaries & Web Chat | Custom OpenAI Base URL → `http://127.0.0.1:8090/v1/chat/completions` | ★★★ **Best Next Target** ("One field, huge reach") |
| 3 | **Raycast** | System-Wide Desktop Shortcuts & Commands | AI → Custom Provider / Ollama host `:8090` | ★★★ Top desktop shortcut launcher |
| 4 | **Home Assistant** | Smart Home Voice Control / Conversation Agent | Native Ollama Integration → `http://127.0.0.1:8090` | ★★★ "Talk to your house" (Most impressive voice demo) |
| 5 | **Joplin** | Open-Source Note Chat & Summaries | Plugins (Jarvis, NoteLLM) → `:8090/v1` | ★★ Non-Obsidian markdown notebooks |
| 6 | **Logseq** | Privacy-First Outliner Writing Assistant | Plugins (`logseq-copilot`) → `:8090/v1` | ★★ Block-level outliner summaries |
| 7 | **Thunderbird** | Private Email Reply Drafting & Thread Summaries | Extension ("Ollama Compose") → `:8090` | ★★ Right-click private inbox triage |
| 8 | **AnythingLLM** | Turnkey Private Document Chat & Workspaces | LLM Provider → Generic OpenAI `:8090/v1` | ★★ Isolated PDF & folder Q&A |
| 9 | **Enchanted / Msty** | Native Desktop & Mobile Chat Clients | Ollama Host → `http://127.0.0.1:8090` | ★★ Friendly desktop frontends |
| 10 | **Claudia** | Local "Second Brain" Chief-of-Staff Assistant | `.claude/settings.local.json` → `:8090` | ★★ 3D `/brain` graph visualizer & MCP daemon |

---

## 4. The Three-Tier Delivery Model for Non-Technical Users

To ensure regular users never have to edit YAML configuration files or manage loopback ports, Token Buffet implements a three-tier delivery framework:

- **Delivery Tier A (Configure Existing App — Lowest Effort)**: Re-pointing an existing, routable third-party app's endpoint field directly to port 8090 (e.g., Brave Leo, Raycast, Home Assistant, Joplin, Thunderbird, AnythingLLM, Open WebUI, or Jan).
- **Delivery Tier B (Build Thin Friendly Integration)**: Creating a lightweight local script or bot bridge where no native consumer app exposes an endpoint (e.g., a Telegram messaging bot bridge or a background IMAP draft script for email).
- **Delivery Tier C (Cloud-Locked)**: Explicitly documenting non-routable tools so users understand system boundaries.

**Dashboard Orchestration (`http://127.0.0.1:8080`)**

Non-technical users manage these connections through the **Control Center Dashboard**. The dashboard's 3-step **Add-an-App Wizard** under the **My Apps** tab guides users through selecting an app type, choosing a workspace folder via a visual directory picker, and picking plain-language template presets (such as *"Research papers"* or *"Daily notes"*). Under the hood, the dashboard executes `integrations/register.py` and pushes updated configuration files directly into target client software.

---

## 5. Primary Consumer Demonstration Stories

Token Buffet focuses its consumer rollout around four high-value demonstration stories:

1. **Unified Cross-Provider Chat (Open WebUI / Jan / Msty)**: Pointed at `http://127.0.0.1:8090/v1`, Open WebUI or desktop frontends provide a single ChatGPT-style interface with multi-user RAG, document uploads, and web search, while Token Buffet load-balances prompts across active subscriptions in the background.
2. **Personal Knowledge & Journaling (Obsidian LLM Wiki / Plume)**: Combines Obsidian's `karpathywiki` engine with local markdown notebooks, turning daily notes and meeting transcripts into interconnected topic pages with `[[wikilinks]]`.
3. **Private Email Triage (Thunderbird + Ollama Compose)**: Allows users to right-click incoming messages in Mozilla Thunderbird to generate contextual reply drafts, summarize 20-message email chains into 3-bullet decision digests, and assign color tags locally without exposing sensitive correspondence to commercial cloud trackers.
4. **Smart Home Autonomy (Home Assistant)**: Leverages Home Assistant's native Ollama integration to evaluate natural-language questions about household states (*"Did anyone leave the garage door open?"*) and trigger complex home automations completely offline.

## Raycast: System-wide AI commands

In the strategy outlined for the Token Buffet AI Subscription Hub, Raycast represents a high-priority integration target for bringing subscription-powered AI to desktop shortcuts and launcher commands.

By connecting Raycast’s native local AI features to Token Buffet’s central router on port 8090, users can execute system-wide AI commands across macOS and Windows—drawing directly from active Claude Code, Codex CLI, Grok Build, and Gemini Antigravity subscriptions without paying per-token developer API fees.

## 1. Capabilities & System-Wide Desktop Reach

Raycast is a keyboard-driven desktop launcher and productivity assistant for macOS and Windows. Its built-in AI capabilities allow users to trigger system-wide commands from anywhere in the operating system without opening a web browser:

- **Instant Hotkey Actions**: Summarizing selected text, rewriting prose, explaining code snippets, or translating text via global desktop shortcuts.
- **Custom Quick AI Commands**: Running user-defined prompt templates and scripts directly from the Raycast launcher overlay.
- **Context-Aware Operations**: Generating quick answers and inline completions using clipboard contents or highlighted text across any active desktop application.

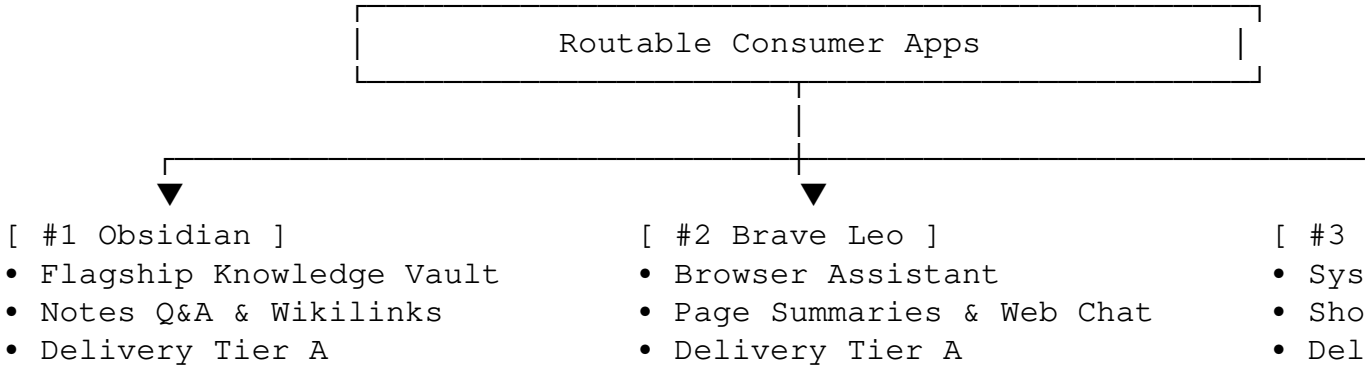
## 2. Technical Wiring & The Universal Config Rule

Connecting Raycast to Token Buffet leverages Raycast's built-in Local Models / Custom Provider support:

- **Endpoint**: Inside Raycast settings (AI -> Local Models / Custom Provider), point the Ollama server/host URL to http://127.0.0.1:8090
- **Model Name**: Enter one of Token Buffet's abstract semantic performance tiers—fast (Quick), medium (Balanced), or reasoning (Best). The router automatically populates dropdowns via /v1/models or /api/tags and dispatches requests to whichever connected provider currently holds the highest available quota headroom.
- **API Key**: Enter any dummy placeholder string (e.g., ollama), as local request authentication is handled internally by the bridge.

## 3. Position in the Context of Potential Consumer Integrations

Raycast occupies a prominent place in Token Buffet's consumer expansion strategy across four key dimensions:




### A. Ranked #3 on the Consumer Roadmap

On Token Buffet's Top 10 consumer routing roadmap, Raycast is ranked #3 overall (behind Obsidian at #1 and Brave Browser Leo at #2) and earns a top-tier ★★★ fit rating. It is recognized as the premier tool for delivering quick, system-wide desktop accessibility.

### B. Delivery Tier A (Lowest Effort)

Token Buffet categorizes consumer integrations into three implementation tiers:

- **Delivery Tier A (Configure Existing App)**: Re-pointing an endpoint setting in an existing third-party app with zero custom code.
- **Delivery Tier B (Thin Friendly Integration)**: Building a lightweight local script or bot bridge where no native consumer app exists.
- **Delivery Tier C (Cloud-Locked)**: Documenting non-routable tools to set clear user expectations.

Raycast is classified under Delivery Tier A, as connecting it requires only a quick endpoint edit inside Raycast's existing settings menu.

### C. Honest Boundary: Routable vs. Cloud-Locked

The documentation establishes a strict distinction between vendor-locked platforms and customizable desktop tools:

- **Cloud-Locked Platforms (Cannot Route)**: Web platforms like ChatGPT web, Gemini web, Claude.ai web, Perplexity, Notion AI, Grammarly, and Canva Magic do not allow changing their API endpoints and cannot be routed to local bridges.
- **The Routable Universe**: Token Buffet focuses on tools like Raycast that explicitly feature BYOM (Bring Your Own Model) or local Ollama endpoint fields, providing everyday desktop AI capabilities backed by subscription quotas.

**D. Value Proposition for Desktop Users**

By connecting Raycast to port 8090, non-technical users gain instant, system-wide desktop access to flat-rate subscription AI. Rather than paying per-token API bills, every desktop shortcut or launcher query is dynamically routed to the best available provider subscription (Claude, GPT, Gemini, or Grok) based on task complexity and remaining account headroom.

## Thunderbird: Private email drafting

In the larger context of the Token Buffet AI Subscription Hub, Thunderbird represents a key consumer target for private email drafting, reply generation, and inbox summarization.

While developer tools focus on coding and terminal automation, integrating Thunderbird demonstrates how flat-rate command-line interface (CLI) subscriptions—such as Claude Code, Codex CLI, Grok Build, and Gemini Antigravity—can power everyday communication tasks for regular, non-technical users without sending email data to vendor clouds or incurring per-token API bills.

Below is a detailed breakdown of what the sources state regarding Thunderbird's technical setup, implementation workflows, roadmap placement, and strategic privacy value.

## 1. Capabilities & Integration via "Ollama Compose"

Thunderbird is a widely used, open-source desktop email client. Within Token Buffet's consumer architecture, it delivers private email assistance through third-party extension bridges:

- **Email Reply Drafting**: Right-clicking an incoming email message inside Thunderbird invokes an extension (such as "Ollama Compose" or AI Emaily) to draft a contextual reply.
- **Inbox & Message Summarization**: Summarizing long email threads, extracting key action items, and adjusting tone directly within the message composition window.
- **Local Communication Privacy**: Email text and sensitive correspondence remain on the user's local machine or travel securely through local CLI proxy tunnels rather than being analyzed by cloud-locked web platforms or third-party tracking services.

## 2. Technical Wiring & The Universal Configuration Rule

Connecting Thunderbird to Token Buffet follows the platform's Universal Configuration Rule for Ollama-compatible applications:

- **Endpoint**: Inside the "Ollama Compose" extension settings, set the server host address to http://127.0.0.1:8090 (Ollama host format).
- **Model Name**: Enter one of Token Buffet's abstract semantic performance tiers—fast (Quick), medium (Balanced), or reasoning (Best). The Smart Router advertises these

tiers via /api/tags and automatically dispatches email prompts to whichever connected subscription backend currently holds the highest remaining quota headroom.
- **API Key**: Enter any dummy placeholder string (e.g., ollama), as request authentication is handled internally by the bridge.

## 3. Implementation Workflows: Tier A vs. Tier B

Token Buffet categorizes consumer application delivery into three implementation tiers based on development effort and value:

- **Delivery Tier A (Lowest Effort — Native Extension)**: Re-pointing the endpoint setting inside Thunderbird's "Ollama Compose" extension to port 8090. This provides instant, right-click reply drafting with zero custom code.
- **Delivery Tier B Fallback (Thin Script Bridge)**: For environments where extensions are not used, Token Buffet proposes a lightweight local IMAP background script. The script fetches unread emails, routes them through port 8090, and automatically saves AI-generated reply drafts directly into the user's IMAP Drafts folder.

## 4. Position on the Consumer Routing Roadmap

Within Token Buffet's strategic roadmap for non-technical everyday software, Thunderbird occupies a distinct position:

- **Ranked #7 Overall (★★ Fit Rating)**: On the Top 10 consumer routable targets table, Thunderbird is ranked #7 with a ★★ fit rating for "everyday email" utility.

#5 in Development Priority: It is positioned #5 in the recommended integration priority sequence for pre-configured integrations/ app types (following #1 Obsidian, #2 Brave Browser Leo, #3 Home Assistant, and #4 Joplin/Logseq).

- **Flagship Consumer Story**: Thunderbird forms one of Token Buffet's three core consumer demonstration stories alongside Unified Chat (Open WebUI) and Journaling (Plume / Obsidian Wiki).
- **Summary**: Routable Consumer App Comparison

| Priority | Application | Category / Feature | Connection Method | Delivery Tier | Strategic Fit |
|---|---|---|---|---|---|
| | Obsidian | Knowledge Vault & Wikilinks | karpathywiki plugin → :8090 | Tier A | ★★★ Flagship (Completed) |
| | Brave Leo | Browser Page Summaries | BYOM Settings → :8090/v1 | Tier A | ★★★ Top Browser Target |
| | Raycast | Desktop Shortcuts & Commands | Custom Provider → :8090 | Tier A | ★★★ System Shortcuts |
| | Home Assistant | Voice Control & Smart Home Agent | Native Ollama Integration → :8090 | Tier A | ★★★ Voice Assistant |
| 5/6 | Joplin / Logseq | Note Chat & Outlining | Plugin Settings → :8090/v1 | Tier A | ★★ Non-Obsidian Notes |
| | Thunderbird | Private Email Drafting | "Ollama Compose" → :8090 | Tier A / B | ★★ Everyday Private Email |

## Joplin and Logseq: Note chat and summaries

In the larger context of the Token Buffet AI Subscription Hub, Joplin and Logseq represent key consumer integrations for note-taking, document summarization, and interactive note chat. While Obsidian serves as Token Buffet's flagship knowledge integration, Joplin and Logseq extend subscription-powered AI (Claude, ChatGPT, Gemini, and Grok) to individuals who prefer open-source note-taking tools or privacy-focused outliner workflows.

## 1. Joplin: Note Chat & Summaries for Open-Source Note-Taking

Joplin is a widely used open-source note-taking and markdown application. Within Token Buffet's consumer architecture, it provides local note chat and automated summarization capabilities:

- **Plugins & Extensions**: Joplin connects to local LLM backends through community plugins such as Jarvis, NoteLLM, or the Joplin AI Agent.
- **Core Capabilities**: Enables users to chat directly with their notes, draft prose, summarize lengthy documents, and extract action items without paying per-token API fees or buying separate developer keys.
- **Connection Endpoint**: The plugin's base URL is set to Token Buffet's OpenAI-compatible endpoint at http://127.0.0.1:8090/v1, with the target model set to one of Token Buffet's abstract semantic performance tiers.

## 2. Logseq: AI-Assisted Outlining & Knowledge Synthesis

Logseq is a privacy-first, open-source outliner and knowledge-management application. It integrates AI capabilities directly into block-based outliner workflows:

- **Plugins & Extensions**: Connects via plugins such as logseq-copilot or third-party local AI integration plugins.
- **Core Capabilities**: Delivers block-level writing assistance, document summarization, note chat, and automated concept synthesis directly within structured outline nodes.
- **Connection Endpoint**: The plugin setting points to Token Buffet's Ollama or OpenAI-compatible endpoint at http://127.0.0.1:8090/v1.

## 3. The Universal Configuration Rule & Delivery Model

Both Joplin and Logseq connect to Token Buffet seamlessly by adhering to the platform's Universal Configuration Rule for routable applications:

- **Endpoint**: Set the plugin's base URL / server host to http://127.0.0.1:8090 (for Ollama host formats) or http://127.0.0.1:8090/v1 (for OpenAI-compatible formats).
- **Model Name**: Select an abstract semantic model tier—fast (Quick), medium (Balanced), or reasoning (Best). The local router populates dropdowns cleanly via /v1/models or /api/tags and dispatches prompts based on available quota headroom or user preferences.
- **API Key**: Use any dummy placeholder string (e.g., ollama).

Delivery Tier Classification

Token Buffet categorizes consumer app delivery into three implementation tiers based on development effort and value:

- **Delivery Tier A (Lowest Effort)**: Re-pointing an endpoint field in an existing, routable third-party app.
- **Delivery Tier B (Thin Friendly Integration)**: Building a lightweight local script or bot where no native consumer app exists.
- **Delivery Tier C (Cloud-Locked)**: Documenting non-routable tools to set clear user expectations.

Both Joplin and Logseq belong to Delivery Tier A, requiring zero custom code creation—simply re-pointing the plugin's API base URL field to port 8090.

## 4. Position on the Consumer Routing Roadmap

Within Token Buffet's strategic roadmap for non-technical users, Joplin and Logseq fulfill a distinct role:

- **Roadmap Rankings**: On the Top 10 consumer routable targets list, Joplin is ranked #5 and Logseq is ranked #6, both earning a ★★ fit rating.
- **Strategic Value**: They are positioned #4 in the recommended integration priority sequence (following #1 Obsidian, #2 Brave Browser Leo, and #3 Home Assistant).
- **Rounding Out the Notes Story**: Their primary strategic purpose is to "round out the notes story for non-Obsidian users". By supporting Joplin and Logseq alongside Obsidian, Token Buffet ensures that users across different note-taking paradigms (standard markdown notebooks, outliners, and Karpathy-style wikis) can leverage flat-rate subscription AI quotas across their personal knowledge bases.